

---

# COIMBATORE INSTITUTE OF TECHNOLOGY

Affiliate by Anna University – cbe -04.



## FULL STACK APPLICATION DEVELOPMENT RECORD

|              |   |                      |
|--------------|---|----------------------|
| NAME         | : | Aanirudh.P           |
| BRANCH       | : | MSc.SOFTWARE SYSTEMs |
| SUBJECT CODE | : | 20MSS51              |
| YEAR         | : | 3 <sup>rd</sup> year |
| DATE         | : | 16/10/2025           |

---

## INDEX

| Ex No | Date       | Index                | Marks |
|-------|------------|----------------------|-------|
| 1     | 20.06.2025 | Type Script          |       |
| 2     | 27.06.2025 | Data Binding         |       |
| 3     | 03.07.2025 | Angular Components   |       |
| 4     | 03.07.2025 | Angular Directives   |       |
| 5     | 04.07.2025 | Angular Routing      |       |
| 6     | 25.07.2025 | Angular Api          |       |
| 7     | 25.07.2025 | Angular Services     |       |
| 8     | 01.08.2025 | React Components     |       |
| 9     | 01.08.2025 | React States & props |       |
| 10    | 01.08.2025 | React Rendering      |       |
| 11    | 25.08.2025 | React Routing        |       |
| 12    | 10.09.2025 | Node-js Mysql        |       |
| 13    | 11.09.2025 | React-Node Mysql     |       |
| 14    | 25.09.2025 | Django 1             |       |
| 15    | 25.09.2025 | Django 2             |       |

EX NO : 1

## **TYPE SCRIPT**

### **Aim :**

- To write TypeScript programs that perform arithmetic operations on arrays, find the length of a string, and categorize numbers based on given conditions.

---

### **Description :**

This exercise involves implementing three TypeScript programs:

- The first program performs arithmetic operations (addition, subtraction, multiplication, division) on corresponding elements of four numeric arrays.
- The second program calculates the length of a given phrase using TypeScript string properties.
- The third program determines if a number is an even positive number, an odd positive number, or a negative number using conditional statements.

### **SOURCE CODE :**

#### **1. Arithmetic Operations Using Four Arrays**

**Typescript -**

```
const arr1: number[] = [1, 2, 3, 4];
```

```
const arr2: number[] = [5, 6, 7, 8];
```

```
const arr3: number[] = [9, 10, 11, 12];
```

```
const arr4: number[] = [13, 14, 15, 16];
```

```
const sumResult = arr1.map((v, i) => v + arr2[i] + arr3[i] + arr4[i]);
```

```
console.log("Sum:", sumResult);
```

```
const diffResult = arr1.map((v, i) => v - arr2[i] - arr3[i] - arr4[i]);  
console.log("Difference:", diffResult);
```

```
const prodResult = arr1.map((v, i) => v * arr2[i] * arr3[i] * arr4[i]);  
console.log("Product:", prodResult);
```

```
const divResult = arr1.map((v, i) => v / arr2[i] / arr3[i] / arr4[i]);  
console.log("Division:", divResult);
```

## 2. Find Length of a Phrase

Typescript

```
const phrase: string = "TypeScript: Full Stack Application Development";  
console.log("Length of phrase:", phrase.length);
```

## 3. Categorize Number

Typescript

```
const n: number = 5; // You can change n for testing
```

```
if (n > 0 && n % 2 === 0) {  
    console.log("even positive number");  
} else if (n > 0 && n % 2 !== 0) {  
    console.log("odd positive number");  
} else {  
    console.log("negative number");  
}
```

#### 4. HTML PAGE :

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>FSAD Exercise 1 Results</title>
```

```
</head>
```

```
<body>
```

```
  <h2>Arithmetic Operations Output</h2>
```

```
  <pre id="arithmetic"></pre>
```

```
  <h2>Phrase Length Output</h2>
```

```
  <pre id="length"></pre>
```

```
  <h2>Number Category Output</h2>
```

```
  <pre id="category"></pre>
```

```
<script type="text/javascript">
```

```
  // This would be compiled JS from your TypeScript codes
```

```
  const arr1 = [1, 2, 3, 4], arr2 = [5, 6, 7, 8], arr3 = [9, 10, 11, 12], arr4 = [13, 14, 15, 16];
```

```
  document.getElementById('arithmetic').textContent =
```

```
    'Sum: ' + arr1.map((v, i) => v + arr2[i] + arr3[i] + arr4[i]) + '\n' +
```

```
    'Difference: ' + arr1.map((v, i) => v - arr2[i] - arr3[i] - arr4[i]) + '\n' +
```

```
    'Product: ' + arr1.map((v, i) => v * arr2[i] * arr3[i] * arr4[i]) + '\n' +
```

```
    'Division: ' + arr1.map((v, i) => v / arr2[i] / arr3[i] / arr4[i]);
```

```
  const phrase = "TypeScript: Full Stack Application Development";
```

```
  document.getElementById('length').textContent = "Length of phrase: " + phrase.length;
```

```
  const n = 5;
```

```
document.getElementById('category').textContent =  
    "Number category: " +  
    (n > 0 && n % 2 === 0 ? 'even positive number' :  
    n > 0 && n % 2 !== 0 ? 'odd positive number' : 'negative number');  
</script>  
</body>  
</html>
```

## EX – 2

## DATA BINDING

---

### I MADE A SIMPLE-TO-DO-LIST

#### AIM :

- To develop a simple Angular application demonstrating string interpolation, property binding, event binding, and two-way binding in a real-world scenario.

#### Description

This Angular application is a minimal to-do list that allows users to add and delete tasks. It uses:

- String interpolation to display the number of tasks and greetings dynamically.
- Property binding to enable or disable the 'Add' button based on the input state.
- Event binding to add and remove tasks upon button clicks.
- Two-way binding to synchronize user input in the textbox with the component's task model.

Through this app, students get hands-on experience with the four principal forms of Angular data binding in a cohesive example.

## Steps :

1. Create a new Angular project:

```
ng new todo-app
```

```
cd todo-app
```

```
ng serve
```

Set up app.module.ts:

Typescript

```
import { NgModule } from '@angular/core';
```

```
import { BrowserModule } from '@angular/platform-browser';
```

```
import { FormsModule } from '@angular/forms'; // Required for ngModel
```

```
import { AppComponent } from './app.component';
```

```
@NgModule({
```

```
  declarations: [ AppComponent ],
```

```
  imports: [ BrowserModule, FormsModule ],
```

```
  providers: [],
```

```
  bootstrap: [ AppComponent ]
```

```
})
```

```
export class AppModule {}
```

🔗 Implement data binding in app.component.ts:

🔗 typescript

```
import { Component } from '@angular/core';
```

```
@Component({
```

```
  selector: 'app-root',
```

```
  templateUrl: './app.component.html',
```

```
})
```

```
export class AppComponent {
```

```
task: string = "";
tasks: string[] = [];
user: string = 'Guest';

addTask() {
  if (this.task.trim()) {
    this.tasks.push(this.task);
    this.task = "";
  }
}
```

```
deleteTask(index: number) {
  this.tasks.splice(index, 1);
}
}
```

Set app.component.html:

```
<h2>Welcome, {{ user }}!</h2> <!-- String Interpolation -->
<p>Tasks: {{ tasks.length }}</p> <!-- String Interpolation -->

<input type="text" [(ngModel)]="task" placeholder="Enter new task" />
<button (click)="addTask()" [disabled]="!task.trim()" >Add</button>

<ul>
  <li *ngFor="let t of tasks; let i = index">
    {{ t }}
    <button (click)="deleteTask(i)">Delete</button>
  </li></ul>
```



## Key Angular Data Binding Concepts Used

- `{{ tasks.length }}` and `{{ user }}`: String interpolation
  - `[disabled]="!task.trim()"`: Property binding
  - `(click)="addTask()"`, `(click)="deleteTask(i)"`: Event binding
  - `[(ngModel)]="task"`: Two-way binding
- 

EX NO : 3

# ANGULAR COMPONENTS

## Aim :

To develop an Angular application that demonstrates the creation of 3 or 4 custom components, mapping them to the main application page, and visually displaying their output in a structured layout as shown in the provided diagram.

---

## Description :

This exercise guides you through building a modular Angular application featuring clear separation of concerns via components. The app will contain:

- A top navigation/header component (Menu/Title)
- A sidebar menu component (Menu)
- A central content area (Body)
- An optional account widget component (Account)

These components will be created individually and placed together inside the root component (AppComponent), visually mimicking a typical dashboard or multi-pane web page. This helps in understanding Angular's component-based architecture, reusability, and template inclusion.

---

## Steps

### 1. Create a New Angular Project

```
ng new angular-components-demo
```

```
cd angular-components-demo
```

## 2. Generate Components

```
ng generate component header    # Component 1: Menu/Title
```

```
ng generate component sidebar  # Component 2: Menu
```

```
ng generate component body     # Component 3: Body
```

```
ng generate component account  # Component 4: Account (Optional)
```

## 3. Edit Each Component's Template

**src/app/header/header.component.html**

```
<div style="border:2px solid blue; padding:8px; font-size:1.5em;">
  Component Menu / <strong>title</strong>
</div>
```

**src/app/sidebar/sidebar.component.html**

```
xml
<div style="border:2px solid red; padding:16px; height:300px;">
  Component Menu
</div>
```

**src/app/body/body.component.html**

```
xml
<div style="border:2px solid magenta; padding:32px; min-height:200px;">
  Component Body
</div>
```

**src/app/account/account.component.html**

```
<div style="border:2px solid green; padding:8px;">
  Component Account
</div>
```

## 4. Compose the Main Layout

**src/app/app.component.html**

```
<div style="border:2px solid orange; padding:4px;">
```

```
<div style="display:flex; flex-direction:row;justify-content:space-between;">

  <div style="flex:9;">

    <app-header></app-header>

  </div>

  <div style="flex:2;">

    <app-account></app-account>

  </div>

</div>

<div style="display:flex;">

  <div style="flex:2;">

    <app-sidebar></app-sidebar>

  </div>

  <div style="flex:8;">

    <app-body></app-body>

  </div>

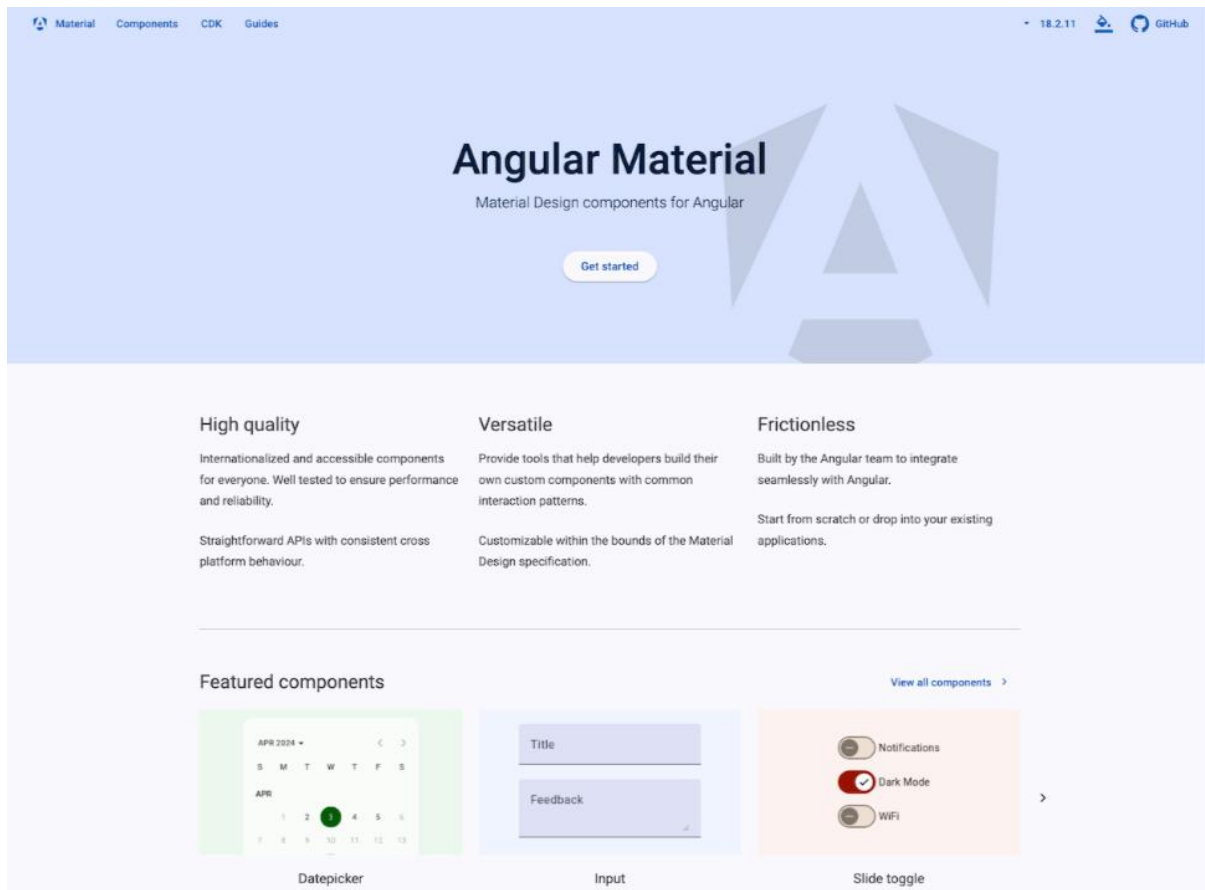
</div>

</div>
```

---

## How It Works

- Each colored block is an Angular component mapped via <app-header>, <app-sidebar>, <app-body>, and <app-account> tags in app.component.html.
- The layout uses basic CSS flexbox for arrangement to match your diagram: header and account on top, menu left, body in the center.
- Run your app using ng serve and visit <http://localhost:4200> to see the result.



## EX NO : 4

## ANGULAR DIRECTIVES

### Aim

- To develop an Angular application (Student Dashboard) that demonstrates the use of structural directives (\*ngIf, \*ngFor, \*ngSwitch) and attribute directives (ngStyle, ngClass) in meaningful real-world scenarios.

### Description

This lab explores Angular's directive system by building a student dashboard. The goal is to:

- List students dynamically with \*ngFor.
- Conditionally show/hide student details such as marks or attendance using \*ngIf.
- Display badges based on each student's performance with \*ngSwitch.
- Use ngStyle to highlight top performers (e.g., highest mark gets bold or colored background).
- Apply ngClass to color-code each row: green for pass, red for fail.

By implementing this dashboard, you will learn how structural and attribute directives can shape both the content and styling of Angular components in response to dynamic data.

## Steps and Example Code

---

### 1. Generate a Student Dashboard Component

bash

ng generate component student-dashboard

### 2. Sample TypeScript (student-dashboard.component.ts)

typescript

```
import { Component } from '@angular/core';
```

```
interface Student {
```

```
  name: string;
```

```
  mark: number;
```

```
  attendance: number;
```

```
}
```

```
@Component({
```

```
  selector: 'app-student-dashboard',
```

```
  templateUrl: './student-dashboard.component.html'
```

```
})
```

```
export class StudentDashboardComponent {
```

```
  showDetails = true;
```

```
  students: Student[] = [
```

```
    { name: 'Alice', mark: 92, attendance: 95 },
```

```
    { name: 'Bob', mark: 78, attendance: 88 },
```

```
    { name: 'Carol', mark: 84, attendance: 90 },
```

```
    { name: 'David', mark: 55, attendance: 60 } ];
```

```
  performanceLevel(mark: number) {
```

```

    if (mark >= 85) return 'Excellent';
    if (mark >= 70) return 'Good';
    return 'Average';
}

```

```

isTop(mark: number) {
    return mark === Math.max(...this.students.map(s => s.mark));
}

```

```

isPass(mark: number) {
    return mark >= 60;
}
}

```

### 3. HTML Template (student-dashboard.component.html)

```

<button (click)="showDetails = !showDetails">Toggle Details</button>
<table>
  <tr>
    <th>Name</th>
    <th>Performance</th>
    <th *ngIf="showDetails">Mark</th>
    <th *ngIf="showDetails">Attendance</th>
  </tr>
  <tr *ngFor="let student of students"
    [ngStyle]="isTop(student.mark) ? {'font-weight':'bold', 'background':'#e0ffe0'} : {}"
    [ngClass]="isPass(student.mark) ? 'pass-row' : 'fail-row'">
    <td>{{student.name}}</td>
    <td>
      <span [ngSwitch]="performanceLevel(student.mark)">
        <span *ngSwitchCase="'Excellent'"> 🏆 Excellent</span>
        <span *ngSwitchCase="'Good'"> 👍 Good</span>
        <span *ngSwitchDefault> 😊 Average</span>
      </span>
    </td>
  </tr>
</table>

```

```

</span>
</td>
<td *ngIf="showDetails">{{student.mark}}</td>
<td *ngIf="showDetails">{{student.attendance}}%</td>
</tr>
</table>

```

#### 4. CSS (student-dashboard.component.css)

CSS

```

.pass-row { background: #d0ffd0; }
.fail-row { background: #ffd0d0; }
table { border-collapse: collapse; width: 70%; }
th, td { border: 1px solid #aaa; padding: 8px; text-align: left; }

```

### Summary of Directives Used

- `*ngFor`: loops through the students array to create one row per student.
- `*ngIf`: controls whether detailed columns (mark, attendance) are shown.
- `*ngSwitch`: displays different badges/messages for each student's performance.
- `[ngStyle]`: highlights the top performer with custom styles.
- `[ngClass]`: applies a green or red background to each row based on pass/fail.



## Student Dashboard

📁 Assignments

📝 Notes

📅 Events

CLASSES

Current Semester

Managerial Accounting

Monday

09:00 AM - 11:30 AM

Business Economics

Tuesday

10:00 - 11:00 AM

View Calendar

UPCOMING EVENTS

Final Exam Week

August 29, 2023 → September 4, 2023

Leadership Workshop

October 16, 2023

Book Fair

October 27, 2023

ASSIGNMENTS

Create a presentation on ethical leadership

Deadline: August 26, 2023

Not started

**EX NO : 5**

# **REACT ROUTING**

## **Aim**

To develop a complete Angular web application using **routing** and **navigation utilities**, allowing users to move between different components/pages through an interactive menu.

---

## **Description**

This Angular website contains several feature components (e.g., Home, About, Contact, Dashboard) and enables users to switch between these views via a menu using Angular's built-in routing system. This approach demonstrates:

---

- How to structure a multi-page single-page application (SPA) using Angular Router.
  - Dynamic, component-based page navigation without full-browser reloads.
  - Practical use of router utilities (RouterModule, routerLink, route definitions, etc.).
- 

## **Steps & Source Code**

### **1. Create a New Angular Project**

```
ng new angular-routing-demo
```

```
cd angular-routing-demo
```

### **2. Generate Feature Components**

```
ng generate component home
```

```
ng generate component about
```

```
ng generate component contact
```

```
ng generate component dashboard
```

### **3. Configure Routing**

```
src/app/app-routing.module.ts
```

```
typescript
```

```
import { NgModule } from '@angular/core';
```

```
import { RouterModule, Routes } from '@angular/router';
```



```
import { HomeComponent } from './home/home.component';
import { AboutComponent } from './about/about.component';
import { ContactComponent } from './contact/contact.component';
import { DashboardComponent } from './dashboard/dashboard.component';
```

```
const routes: Routes = [
  { path: '', component: HomeComponent },      // default route
  { path: 'about', component: AboutComponent },
  { path: 'dashboard', component: DashboardComponent },
  { path: 'contact', component: ContactComponent }
];
```

```
@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
```

```
export class AppRoutingModule { }
```

**src/app/app.module.ts**

typescript

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppRoutingModule } from './app-routing.module'; // Import routing
import { AppComponent } from './app.component';
import { HomeComponent } from './home/home.component';
import { AboutComponent } from './about/about.component';
import { ContactComponent } from './contact/contact.component';
import { DashboardComponent } from './dashboard/dashboard.component';
```

```
@NgModule({
  declarations: [
    AppComponent,
```

```

    HomeComponent,
    AboutComponent,
    ContactComponent,
    DashboardComponent
  ],
  imports: [BrowserModule, AppRoutingModule],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }

```

#### 4. Create Application Menu (AppComponent)

##### src/app/app.component.html

```

<nav style="background: #f0f0f0; padding: 1em;">
  <a routerLink="/" routerLinkActive="active" [routerLinkActiveOptions]="{ exact: true }">Home</a>
  |
  <a routerLink="/about" routerLinkActive="active">About</a> |
  <a routerLink="/dashboard" routerLinkActive="active">Dashboard</a> |
  <a routerLink="/contact" routerLinkActive="active">Contact</a>
</nav>

<div style="margin-top: 2em;">
  <router-outlet></router-outlet>
</div>

```

##### src/styles.css

```

css

.active {
  font-weight: bold;
  color: green;
  text-decoration: underline;
}

nav a {

```

```
margin-right: 12px;  
}
```

## 5. Sample Content for Feature Components

**src/app/home/home.component.html**

```
<h2>Welcome to Home Page!</h2>  
<p>This is the Angular Routing Demo Home.</p>
```

**src/app/about/about.component.html**

```
<h2>About Us</h2>  
<p>Learn more about our Angular site and features here.</p>
```

**src/app/dashboard/dashboard.component.html**

```
<h2>Dashboard</h2>  
<p>Secure area for logged in users and admin features.</p>
```

**src/app/contact/contact.component.html**

```
<h2>Contact</h2>  
<p>You can reach us via the form below.</p>
```

## 6. Run the Application

bash

ng serve

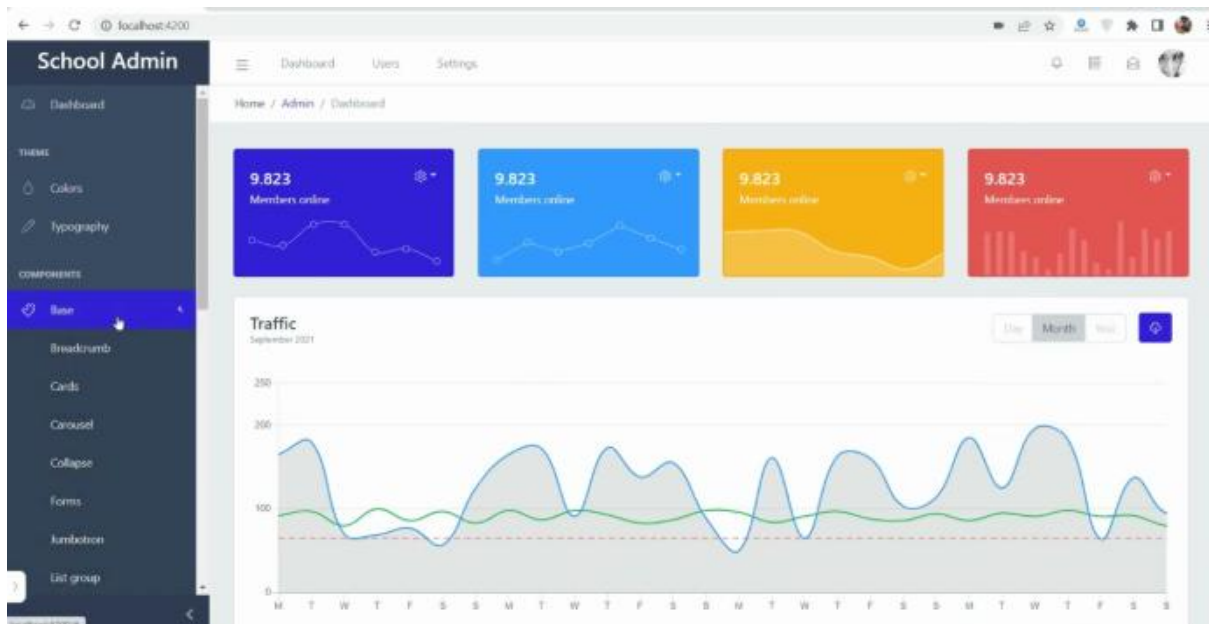
Open <http://localhost:4200> and navigate between pages via the menu.

---

## Routing Utilities Used

---

- **RouterModule/Routes:** To define paths and matching components.
  - **routerLink:** Binds menu links directly to Angular navigation.
  - **routerLinkActive:** Highlights the active page link in the menu.
  - **router-outlet:** Placeholder for routed components displayed on navigation.
-



---

**EX NO : 6**

## **ANGULAR API**

### **Aim**

- To build an Angular application that fetches, displays, formats, updates, and deletes live JSON data from an API endpoint, demonstrating practical front-end data handling in Angular.
- 

### **Description**

This example shows how an Angular component interacts with a REST API (such as <https://jsonplaceholder.typicode.com/users>) to perform all CRUD operations (except "create") on live user data. The app lists users (displayed in a formatted table), allows in-browser data updates, and supports deletion of a user, all using standard Angular HTTP utilities and component logic.

---

## Steps & Sample Code

### 1. Create Angular Project and Service

```
ng new angular-api-demo
cd angular-api-demo
ng generate component users
ng generate service user
```

### 2. Import HttpClientModule

In `app.module.ts`:

Typescript

```
import { HttpClientModule } from '@angular/common/http';
```

```
@NgModule({
  // ...
  imports: [BrowserModule, HttpClientModule],
  // ...
})
export class AppModule {}
```

### 3. User Service (`user.service.ts`)

Typescript

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
```

```

@Injectables({ providedIn: 'root' })
export class UserService {
  private apiUrl = 'https://jsonplaceholder.typicode.com/users';

  constructor(private http: HttpClient) {}

  getUsers(): Observable<any[]> {
    return this.http.get<any[]>(this.apiUrl);
  }

  updateUser(user: any): Observable<any> {
    return this.http.put(`${this.apiUrl}/${user.id}`, user);
  }

  deleteUser(id: number): Observable<any> {
    return this.http.delete(`${this.apiUrl}/${id}`);
  }
}

```

## 4. Component Logic (users.component.ts)

Typescript

```

import { Component, OnInit } from '@angular/core';
import { UserService } from '../user.service';

@Component({

```

```
    selector: 'app-users',
    templateUrl: './users.component.html'
  })
  export class UsersComponent implements OnInit {
    users: any[] = [];
    editId: number | null = null;
    newName: string = '';

    constructor(private userService: UserService) {}

    ngOnInit() {
      this.userService.getUsers().subscribe(data => this.users = data);
    }

    startEdit(user: any) {
      this.editId = user.id;
      this.newName = user.name;
    }

    saveEdit(user: any) {
      const updated = {...user, name: this.newName};
      this.userService.updateUser(updated).subscribe(res => {
        user.name = this.newName; // Update UI after successful update
        this.editId = null;
      });
    }
  }
```

```

deleteUser(user: any) {
  this.userService.deleteUser(user.id).subscribe(res => {
    this.users = this.users.filter(u => u.id !== user.id);
  });
}
}

```

## 5. Component HTML (users.component.html)

```
<h2>User List (Live API Demo)</h2>
```

```
<table border="1">
```

```
<tr>
```

```
<th>ID</th><th>Name</th><th>Email</th><th>Actions</th>
```

```
</tr>
```

```
<tr *ngFor="let user of users">
```

```
<td>{{user.id}}</td>
```

```
<td *ngIf="editId !== user.id">{{ user.name }}</td>
```

```
<td *ngIf="editId === user.id">
```

```
<input [(ngModel)]="newName"/>
```

```
</td>
```

```
<td>{{ user.email }}</td>
```

```
<td>
```

```
<button *ngIf="editId !== user.id" (click)="startEdit(user)">Edit</button>
```

```
<button *ngIf="editId === user.id" (click)="saveEdit(user)">Save</button>
```

```
<button (click)="deleteUser(user)">Delete</button>
```

```
</td>
```

```
</tr>
```



</table>

## How Each Requirement is Met

- **Display the data content:** Uses `getUsers()` to list table data from API.
- **Format the data:** Table layout with separate columns; conditional rendering for edit mode.
- **Update the data:** Edit button turns row into input, Save calls `updateUser()`.
- **Delete the data:** Clicking Delete removes user from both API/result set.

≡ LEAVE

Check-in

0

Adam ▾

All Assigned Leaves

Filter

Assign

| Employee                                                                                             | Leave Type   | Available Days | Carryforward Days | Total Leave Days | Assigned Date | Actions                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------|--------------|----------------|-------------------|------------------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  Alexander Kim    | Casual Leave | 12             | 0                 | 12               | May 3, 2023   |   |
|  Alexander Kim    | Sick Leave   | 6              | 0                 | 6                | May 3, 2023   |   |
|  Alexander Wilson | Casual Leave | 12             | 0                 | 12               | May 3, 2023   |   |
|  Alexander Wilson | Sick Leave   | 10             | 0                 | 10               | May 3, 2023   |   |
|  Amelia Brown     | Casual Leave | 12             | 0                 | 12               | May 3, 2023   |   |
|  Amelia Brown     | Sick Leave   | 10             | 0                 | 10               | May 3, 2023   |   |
|  Amy Chen         | Casual Leave | 12             | 0                 | 12               | May 3, 2023   |   |
|  Amy Chen         | Sick Leave   | 10             | 0                 | 10               | May 3, 2023   |   |
|  Andrew Chen      | Casual Leave | 12             | 0                 | 12               | May 3, 2023   |   |

## **Aim**

To create and inject multiple Angular services in at least two components, and display the output to illustrate service usage, dependency injection, and shared logic/data.

---

## **Description**

This application defines two Angular services (MessageService and DataService). These services are injected into two different components (HomeComponent and ProfileComponent). The services provide independent functionalities—one for messaging and one for sharing data—which can be reused and called from the components. This example demonstrates the power and convenience of Angular's dependency injection pattern.

---

## **Steps & Source Code**

### **1. Generate Services and Components**

```
ng new angular-service-demo
cd angular-service-demo
ng generate service message
ng generate service data
ng generate component home
ng generate component profile
```

---

### **2. First Service—MessageService**

```
src/app/message.service.ts
```

## Typescript

```
import { Injectable } from '@angular/core';
```

```
@Injectable({ providedIn: 'root' })
```

```
export class MessageService {
```

```
  getWelcomeMessage(user: string): string {
```

```
    return `Welcome, ${user}!`;
```

```
  }
```

```
  getGoodbyeMessage(user: string): string {
```

```
    return `Goodbye, ${user}.`;
```

```
  }
```

```
}
```

---

### 3. Second Service—DataService

src/app/data.service.ts

typescript

```
import { Injectable } from '@angular/core';
```

```
@Injectable({ providedIn: 'root' })
```

```
export class DataService {
```

```
  private value = 0;
```

```
  getValue(): number {
```

```
    return this.value;
```

```
  }
```

```
  increment(): void {
```

```
    this.value++;
```

```
}  
reset(): void {  
  this.value = 0;  
}  
}
```

---

## 4. Home Component — inject both services

**src/app/home/home.component.ts**

### **Typescript**

```
import { Component } from '@angular/core';  
import { MessageService } from '../message.service';  
import { DataService } from '../data.service';  
  
@Component({  
  selector: 'app-home',  
  templateUrl: './home.component.html'  
})  
export class HomeComponent {  
  user = 'Alice';  
  msg: string;  
  value: number;  
  
  constructor(private msgService: MessageService, private dataService:  
    DataService) {  
    this.msg = this.msgService.getWelcomeMessage(this.user);  
    this.value = this.dataService.getValue();  
  }  
}
```

```
}
```

```
incrementValue() {  
  this.dataService.increment();  
  this.value = this.dataService.getValue();  
}  
}
```

**src/app/home/home.component.html**

```
<h3>Home Component</h3>  
<p>{{ msg }}</p>  
<p>Shared Value: {{ value }}</p>  
<button (click)="incrementValue()">Increment</button>
```

---

## 5. Profile Component — inject both services

**src/app/profile/profile.component.ts**

**Typescript**

```
import { Component } from '@angular/core';  
import { MessageService } from '../message.service';  
import { DataService } from '../data.service';  
  
@Component({
```

```

    selector: 'app-profile',
    templateUrl: './profile.component.html'
  })
  export class ProfileComponent {
    user = 'Bob';
    msg: string;
    value: number;

    constructor(private msgService: MessageService, private dataService:
DataService) {
      this.msg = this.msgService.getGoodbyeMessage(this.user);
      this.value = this.dataService.getValue();
    }

    resetValue() {
      this.dataService.reset();
      this.value = this.dataService.getValue();
    }
  }

```

**src/app/profile/profile.component.html**

```

h3>Profile Component</h3>
<p>{{ msg }}</p>
<p>Shared Value: {{ value }}</p>
<button (click)="resetValue()">Reset</button>

```

---

## 6. Map Both Components in AppComponent

src/app/app.component.html

xml

<app-home></app-home>

<hr>

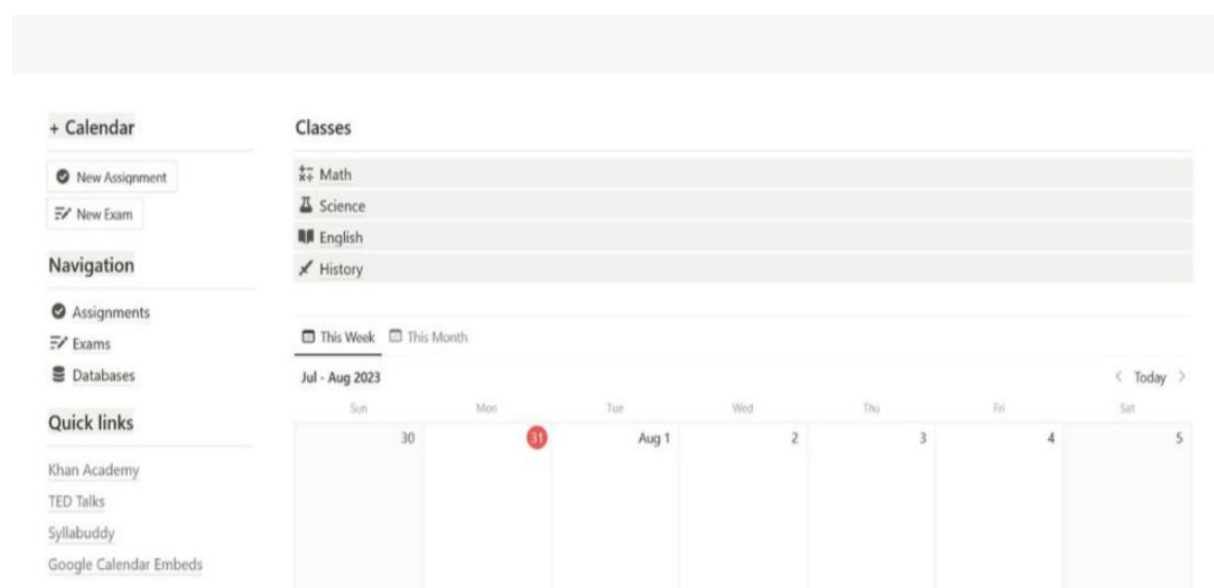
<app-profile></app-profile>

---

## Output Explanation

---

- Both components access different functions from the same injected services.
  - The shared value changes in both components, demonstrating how services act as singleton state/data stores across components.
  - Output appears as messages, values, and buttons in the UI.
- 



---

Ex No: 8

# REACT COMPONENTS

Date: 01.08.2025

---

## AIM:-

Set up a React Project and develop a basic website and demonstrate the following utilities in React JS , JSX Basic data Rendering and Usage of Functional based Components

Note: design interactive neat colored webpages for the above.

Create and call components

1. from parent to child components
2. from separate component files

## DESCRIPTION:-

React uses Node.js to run and manage dependencies.

 **Download from:**

 <https://nodejs.org>

 **Steps:**

1. Visit the above link.
2. Download the **LTS (Long Term Support)** version for your system (Windows/Mac/Linux).
3. Install it by clicking **Next → Next → Finish**.
4. After installation, open **Command Prompt (cmd)** or **PowerShell**, and type:
5. `node -v`
6. `npm -v`

If both show version numbers, you're good to go and create folder and open in code editor



## **CODE:-**

✖ Step 1 — Open VS Code terminal or Command Prompt

Use:

```
npm create vite@latest my-vite-react-app
```

✖ Step 2 — Choose Framework & Variant

Vite will ask:

✓ Select a framework: › React

✓ Select a variant: › JavaScript

(If you want TypeScript, choose “React + TypeScript” instead.)

✖ Step 3 — Go inside your project folder

```
cd my-vite-react-app
```

✖ Step 4 — Install dependencies

```
npm install
```

✖ Step 5 — Run the development server

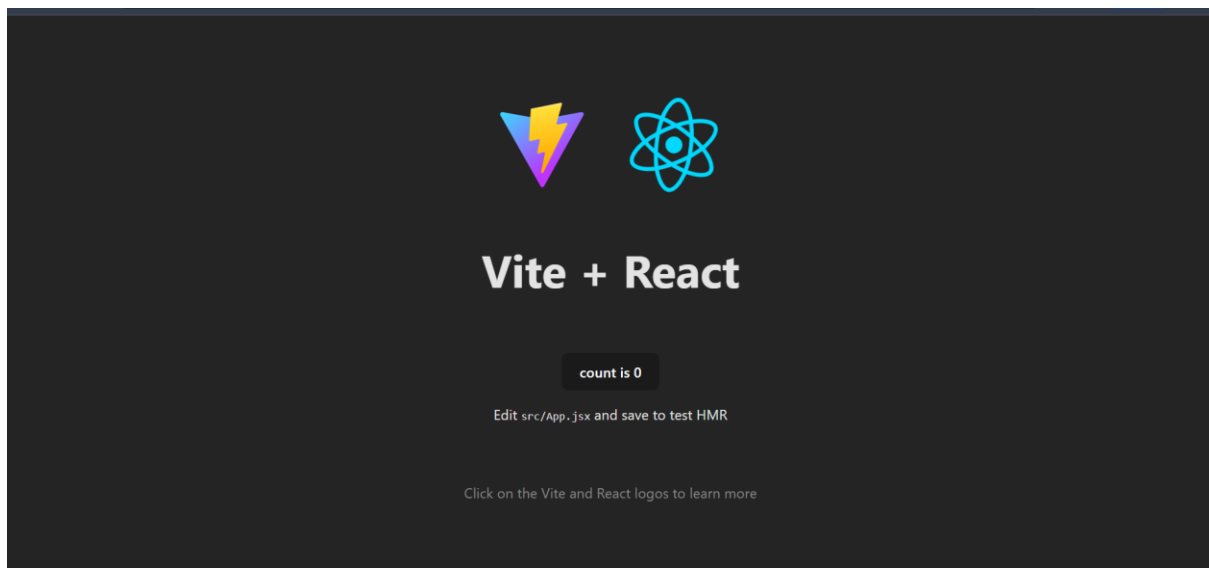
```
npm run dev
```

You'll see output like:

VITE v5.x ready in 300ms

→ Local: <http://localhost:5173/>

Open this link in your browser.



```
VITE v7.1.10 ready in 657 ms
→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

---

Ex No:9

## USAGE OF PROPS

Date:01.08.2025

---

### **AIM:-**

EXERCISE – 09

#### USAGE OF PROPS

Develop a webpage to demonstrate the following using React JS:

- **Creation of a Project**
- **Creation of Function Components**
- **Function Components with props (all available options)**
- **Connect parent & child components**
- **Display the components one by one in the main web page**
- **Design the components with appropriate design using CSS /**
- **Bootstrap.**

#### USAGE OF REACT STATES & HOOKS

Develop a webpage to demonstrate the following using React JS:

- **Create states in 'function based components' using HOOK**

useState and the change the values using (setstate) text  
change event.

## **DESCRIPTION:-**

- The game is built as a functional React component using `useState` and `useEffect` hooks.
- `useState` manages dynamic game data like happiness level, progress, timer, and selected background.
- happiness state tracks how “happy” the character is, increasing when correct actions occur.
- progress state updates a progress bar visually showing game completion.
- timer state controls countdown or duration tracking for time-based events.
- `isPopupVisible` state toggles between the main game and popup screens.
- `selectedImage` state stores which image theme (roman, park, snow, etc.) is currently active.
- `isAnimating` state enables fun animation effects like shaking or glowing when interactions happen.
- `useEffect` handles side effects such as running timers, updating the bar, and triggering win conditions.
- Props (if used for subcomponents like `Popup` or `Child`) pass read-only data such as messages or image names from the parent to child components.
- Button click handlers modify states to control happiness, animation, and game progress interactively.
- Background images (roman.jpg, park.jpg, snow.jpeg, etc.) change based on user actions, giving a dynamic look.
- The game includes visual feedback — confetti, color transitions, or glow effects when happiness reaches a goal.
- `HTML2Canvas` is used to take a screenshot or snapshot of the current screen (extra creative feature).
- Overall, the app combines React state management, props for communication, interactive animations, timers, and visual styling to create a lively and responsive game experience.

## **CODE:-**

```
import React, { useState, createContext, useContext, useEffect, useReducer, useRef,
useMemo, useCallback, memo } from 'react';

import RomanticBg from '../assets/roman.jpg';

import ButterflyBg from '../assets/park.jpg';

import SnowBg from '../assets/snow.jpeg';
```

```

import WindBg from '../assets/winnd.jpg';
import html2canvas from 'html2canvas';
import './Find.css';
import { FaTwitter, FaWhatsapp, FaInstagram, FaTelegram, FaFacebook } from 'react-icons/fa';

const GameContext = createContext();

const settingsReducer = (state, action) => {
  switch (action.type) {
    case 'SET_DIFFICULTY':
      return { ...state, difficulty: action.payload };
    case 'SET_SIZE':
      return { ...state, targetSize: action.payload };
    case 'SET_DURATION':
      return { ...state, gameDuration: action.payload };
    case 'SET_CATEGORY':
      return { ...state, categoryFocus: action.payload };
    case 'SET_THEME':
      return { ...state, themeEffect: action.payload };
    case 'TOGGLE_HUD':
      return { ...state, showHUD: !state.showHUD };
    case 'RESET':
      return action.payload;
    default:
      return state;
  }
};

// Scoring Reducer

```

```
const scoreReducer = (state, action) => {  
  switch (action.type) {  
    case 'HIT': {  
      const newCombo = state.combo + 1;  
      const multiplier = Math.floor(newCombo / 5) + 1;  
      return {  
        ...state,  
        score: state.score + (action.points * multiplier),  
        hits: state.hits + 1,  
        combo: newCombo,  
        categoryScores: {  
          ...state.categoryScores,  
          [action.category]: (state.categoryScores[action.category] || 0) + (action.points *  
multiplier)  
        }  
      };  
    }  
    case 'MISS':  
      return {  
        ...state,  
        misses: state.misses + 1,  
        combo: 0  
      };  
    case 'RESET':  
      return {  
        score: 0,  
        hits: 0,  
        misses: 0,  
        combo: 0,  
        categoryScores: {}  
      };  
  }  
}
```

```

    };

    default:
        return state;
    }
};

// Theme Effect Components

const SnowEffect = memo(() => {
    const [snowflakes, setSnowflakes] = useState([]);

    useEffect(() => {
        const generateSnowflakes = () => {
            const flakes = [];
            for (let i = 0; i < 50; i++) {
                flakes.push({
                    id: i,
                    left: Math.random() * 100,
                    animationDuration: Math.random() * 3 + 2,
                    opacity: Math.random() * 0.6 + 0.4,
                    size: Math.random() * 10 + 5
                });
            }
            setSnowflakes(flakes);
        };

        generateSnowflakes();
    }, []);

```

```

return (
  <div className="snow-effect">
    {snowflakes.map(flake => (
      <div
        key={flake.id}
        className="snowflake"
        style={{
          left: `${flake.left}%`,
          animationDuration: `${flake.animationDuration}s`,
          opacity: flake.opacity,
          width: `${flake.size}px`,
          height: `${flake.size}px`
        }}
      >
        ❄️
      </div>
    ))}
  </div>
);
});

```

```

const WindEffect = memo(() => {
  const [leaves, setLeaves] = useState([]);

  useEffect(() => {
    const generateLeaves = () => {
      const leafSymbols = ['🌿', '🌱', '🌲', '🌴'];
      const newLeaves = [];
      for (let i = 0; i < 25; i++) {

```



```

newLeaves.push({
  id: i,
  symbol: leafSymbols[Math.floor(Math.random() * leafSymbols.length)],
  top: Math.random() * 100,
  animationDuration: Math.random() * 4 + 3,
  opacity: Math.random() * 0.7 + 0.3,
  size: Math.random() * 0.5 + 0.8
});
}
setLeaves(newLeaves);
};

generateLeaves();
}, []);

return (
  <div className="wind-effect">
    {leaves.map(leaf => (
      <div
        key={leaf.id}
        className="wind-leaf"
        style={{
          top: `${leaf.top}%`,
          animationDuration: `${leaf.animationDuration}s`,
          opacity: leaf.opacity,
          fontSize: `${leaf.size}rem`
        }}
      >
        {leaf.symbol}

```

```

    </div>

  )}
</div>

);
});

const ButterflyEffect = memo(() => {
  const [butterflies, setButterflies] = useState([]);

  useEffect(() => {
    const generateButterflies = () => {
      const butterflySymbols = ['🦋', '🦋', '🦋'];
      const newButterflies = [];
      for (let i = 0; i < 15; i++) {
        newButterflies.push({
          id: i,
          symbol: butterflySymbols[Math.floor(Math.random() * butterflySymbols.length)],
          left: Math.random() * 100,
          top: Math.random() * 100,
          animationDuration: Math.random() * 6 + 4,
          opacity: Math.random() * 0.8 + 0.4,
          size: Math.random() * 0.5 + 1
        });
      }
      setButterflies(newButterflies);
    };

    generateButterflies();
  }, []);

```

```

return (
  <div className="butterfly-effect">
    {butterflies.map(butterfly => (
      <div
        key={butterfly.id}
        className="butterfly"
        style={{
          left: `${butterfly.left}%`,
          top: `${butterfly.top}%`,
          animationDuration: `${butterfly.animationDuration}s`,
          opacity: butterfly.opacity,
          fontSize: `${butterfly.size}rem`
        }}
      >
        {butterfly.symbol}
      </div>
    ))}
  </div>
);
});

```

```

const RomanticEffect = memo(() => {
  const [romanticElements, setRomanticElements] = useState([]);

  useEffect(() => {
    const generateRomanticElements = () => {
      const romanticSymbols = ['💖', '🌹', '💕', '❤️', '💞', '💐', '🌺'];
      const newElements = [];

```

```

for (let i = 0; i < 20; i++) {
  newElements.push({
    id: i,
    symbol: romanticSymbols[Math.floor(Math.random() * romanticSymbols.length)],
    left: Math.random() * 100,
    animationDuration: Math.random() * 5 + 3,
    opacity: Math.random() * 0.6 + 0.3,
    size: Math.random() * 0.4 + 0.8,
    delay: Math.random() * 2
  });
}
setRomanticElements(newElements);
};

generateRomanticElements();
}, []);

return (
  <div className="romantic-effect">
    {romanticElements.map(element => (
      <div
        key={element.id}
        className="romantic-element"
        style={{
          left: `${element.left}%`,
          animationDuration: `${element.animationDuration}s`,
          animationDelay: `${element.delay}s`,
          opacity: element.opacity,
          fontSize: `${element.size}rem`

```

```

    }}
  >
    {element.symbol}
  </div>
  )}}
</div>

);
});

```

// Floating Score Component

```

const FloatingScore = memo(({ x, y, points, onRemove }) => {
  useEffect(() => {
    const timer = setTimeout(onRemove, 1000);
    return () => clearTimeout(timer);
  }, [onRemove]);

  return (
    <div
      className="floating-score"
      style={{ left: x, top: y }}
    >
      +{points}
    </div>
  );
});

```

// Circle Component

```

const Circle = memo(({ circle, onCircleClick, targetSize }) => {
  const [isHovered, setIsHovered] = useState(false);

```

```

const handleClick = useCallback((e) => {
  e.preventDefault();
  onCircleClick(circle, e.clientX, e.clientY);
}, [circle, onCircleClick]);

const sizeClass = targetSize === 'small' ? 'circle-small' :
  targetSize === 'large' ? 'circle-large' : 'circle-medium';

return (
  <div
    className={`circle ${sizeClass} ${circle.isBurst ? 'burst' : ''} ${isHovered ? 'wobble' : ''}`}
    style={{
      left: `${circle.position.x}%`,
      top: `${circle.position.y}%`,
      backgroundColor: circle.color
    }}
    onClick={handleClick}
    onMouseEnter={() => setIsHovered(true)}
    onMouseLeave={() => setIsHovered(false)}
  >
    {circle.emoji && (
      <span className="emoji">{circle.emoji}</span>
    )}
  </div>
);
});

// Main Game Component

```

```
const Find = () => {  
  const initialSettings = {  
    difficulty: 'medium',  
    targetSize: 'medium',  
    gameDuration: 60,  
    categoryFocus: 'happy',  
    themeEffect: 'none',  
    showHUD: true  
  };  
  
  const [settings, dispatchSettings] = useReducer(settingsReducer, initialSettings);  
  
  // Background style depends on current themeEffect in settings  
  let backgroundStyle;  
  switch (settings.themeEffect) {  
    case 'romantic':  
      backgroundStyle = { backgroundImage: `url(${RomanticBg})` };  
      break;  
    case 'butterfly':  
      backgroundStyle = { backgroundImage: `url(${ButterflyBg})` };  
      break;  
    case 'snow':  
      backgroundStyle = { backgroundImage: `url(${SnowBg})` };  
      break;  
    case 'wind':  
      backgroundStyle = { backgroundImage: `url(${WindBg})` };  
      break;  
    default:  
      backgroundStyle = { backgroundColor: '#59948f' }; // default color for "none"  
  }  
}
```

```
}
```

```
const [gameState, setGameState] = useState('start'); // start, playing, ended
```

```
const [circles, setCircles] = useState([]);
```

```
const [scoreState, dispatchScore] = useReducer(scoreReducer, {
```

```
  score: 0,
```

```
  hits: 0,
```

```
  misses: 0,
```

```
  combo: 0,
```

```
  categoryScores: {}
```

```
});
```

```
const [timeLeft, setTimeLeft] = useState(60);
```

```
const [floatingScores, setFloatingScores] = useState([]);
```

```
const [particles, setParticles] = useState([]);
```

```
const gameAreaRef = useRef(null);
```

```
const timerRef = useRef(null);
```

```
const spawnTimerRef = useRef(null);
```

```
const audioContextRef = useRef(null);
```

```
const emojiCategories = useMemo(() => ({
```

```
  happy: { emojis: ['😊', '😄', '😁', '😃', '😆', '😅', '😂', '😇', '😊'], color: '#FFD700', points: 10 },
```

```
  sad: { emojis: ['😞', '😓', '😔', '😕', '😖', '😗', '😘', '😙', '😚', '😛'], color: '#4682B4', points: 5 },
```

```
  angry: { emojis: ['😠', '😡', '😢', '😣', '😤', '😥', '😦', '😧', '😨', '😩'], color: '#DC143C', points: 8 },
```

```
  surprise: { emojis: ['😮', '😯', '😰', '😱', '😲', '😳', '😴', '😵', '😶', '😷'], color: '#800080', points: 12 }
```

```
}), []);
```



```
const spawnConfig = useMemo(() => ({
  easy: { interval: 2000, maxCircles: 3 },
  medium: { interval: 1500, maxCircles: 5 },
  hard: { interval: 1000, maxCircles: 7 }
}), []);
```

```
const playSound = useCallback((frequency, duration = 100) => {
  if (!audioContextRef.current) {
    audioContextRef.current = new (window.AudioContext ||
window.webkitAudioContext)();
  }
```

```
  const oscillator = audioContextRef.current.createOscillator();
  const gainNode = audioContextRef.current.createGain();
```

```
  oscillator.connect(gainNode);
  gainNode.connect(audioContextRef.current.destination);
```

```
  oscillator.frequency.value = frequency;
  oscillator.type = 'sine';
```

```
  gainNode.gain.setValueAtTime(0.3, audioContextRef.current.currentTime);
  gainNode.gain.exponentialRampToValueAtTime(0.01,
audioContextRef.current.currentTime + duration / 1000);
```

```
  oscillator.start(audioContextRef.current.currentTime);
  oscillator.stop(audioContextRef.current.currentTime + duration / 1000);
}, []);
```

```
const getRandomPosition = useCallback(() => ({
```

```
x: Math.random() * 85 + 5,  
y: Math.random() * 75 + 10  
}), []);
```

```
const spawnCircle = useCallback(() => {  
  if (circles.length >= spawnConfig[settings.difficulty].maxCircles) return;
```

```
  const categories = Object.keys(emojiCategories);  
  const focusCategory = settings.categoryFocus;
```

```
  const selectedCategory = Math.random() < 0.6 ?  
    focusCategory :  
    categories[Math.floor(Math.random() * categories.length)];
```

```
  const categoryData = emojiCategories[selectedCategory];  
  const emoji = categoryData.emojis[Math.floor(Math.random() *  
categoryData.emojis.length)];
```

```
  const newCircle = {  
    id: Date.now() + Math.random(),  
    category: selectedCategory,  
    emoji: emoji,  
    color: categoryData.color,  
    position: getRandomPosition(),  
    isBurst: false,  
    spawnTime: Date.now()  
  };
```

```
  setCircles(prev => [...prev, newCircle]);
```

```
setTimeout(() => {
  setCircles(prev => {
    const circle = prev.find(c => c.id === newCircle.id);
    if (circle && !circle.isBurst) {
      dispatchScore({ type: 'MISS' });
      playSound(150, 200);
    }
    return prev.filter(c => c.id !== newCircle.id);
  });
}, 3000);

}, [circles.length, spawnConfig, settings.difficulty, settings.categoryFocus, emojiCategories,
getRandomPosition, playSound]);
```

```
const handleCircleClick = useCallback((circle, clientX, clientY) => {
  if (circle.isBurst) return;
```

```
const categoryData = emojiCategories[circle.category];
dispatchScore({
  type: 'HIT',
  points: categoryData.points,
  category: circle.category
});
```

```
playSound(400 + (scoreState.combo * 50), 150);
```

```
const rect = gameAreaRef.current.getBoundingClientRect();
setFloatingScores(prev => [...prev, {
  id: Date.now(),
  x: clientX - rect.left,
  y: clientY - rect.top,
```

```
    points: categoryData.points
  });
```

```
const particleCount = 8;
const newParticles = [];
for (let i = 0; i < particleCount; i++) {
  newParticles.push({
    id: Date.now() + i,
    x: clientX - rect.left,
    y: clientY - rect.top,
    angle: (i * 360) / particleCount,
    color: categoryData.color
  });
}
setParticles(prev => [...prev, ...newParticles]);
```

```
setCircles(prev => prev.map(c =>
  c.id === circle.id ? { ...c, isBurst: true } : c
));
```

```
setTimeout(() => {
  setCircles(prev => prev.filter(c => c.id !== circle.id));
}, 500);
}, [emojiCategories, scoreState.combo, playSound]);
```

```
const removeFloatingScore = useCallback((id) => {
  setFloatingScores(prev => prev.filter(s => s.id !== id));
}, []);
```

```
const startGame = useCallback(() => {  
  setGameState('playing');  
  setTimeLeft(settings.gameDuration);  
  dispatchScore({ type: 'RESET' });  
  setCircles([]);  
  setFloatingScores([]);  
  setParticles([]);  
  
  spawnTimerRef.current = setInterval(spawnCircle,  
spawnConfig[settings.difficulty].interval);
```

```
timerRef.current = setInterval(() => {  
  setTimeLeft(prev => {  
    if (prev <= 1) {  
      setGameState('ended');  
      clearInterval(timerRef.current);  
      clearInterval(spawnTimerRef.current);  
      return 0;  
    }  
    return prev - 1;  
  });  
}, 1000);  
}, [settings, spawnCircle, spawnConfig]);
```

```
const resetGame = useCallback(() => {  
  setGameState('start');  
  setCircles([]);  
  setFloatingScores([]);  
  setParticles([]);  
  dispatchScore({ type: 'RESET' });
```

```
clearInterval(timerRef.current);  
clearInterval(spawnTimerRef.current);  
}, []);
```

```
const happinessPercentage = useMemo(() => {  
  const totalScore = Object.values(scoreState.categoryScores).reduce((sum, score) => sum +  
score, 0);  
  const happyScore = scoreState.categoryScores.happy || 0;  
  return totalScore > 0 ? Math.round((happyScore / totalScore) * 100) : 0;  
}, [scoreState.categoryScores]);
```

```
useEffect(() => {  
  return () => {  
    clearInterval(timerRef.current);  
    clearInterval(spawnTimerRef.current);  
  };  
}, []);
```

```
useEffect(() => {  
  if (particles.length > 0) {  
    const timer = setTimeout(() => {  
      setParticles([]);  
    }, 1000);  
    return () => clearTimeout(timer);  
  }  
}, [particles]);
```

```
const renderThemeEffect = () => {  
  if (gameState !== 'playing') return null;
```

```
switch (settings.themeEffect) {  
  case 'snow':  
    return <SnowEffect />;  
  case 'wind':  
    return <WindEffect />;  
  case 'butterfly':  
    return <ButterflyEffect />;  
  case 'romantic':  
    return <RomanticEffect />;  
  default:  
    return null;  
}  
};
```

```
const contextValue = {  
  settings,  
  dispatchSettings,  
  gameState,  
  scoreState,  
  startGame,  
  resetGame  
};
```

```
return (  
  <GameContext.Provider value={contextValue}>  
    <div className={`game-container ${settings.categoryFocus}-theme`}>  
      {gameState === 'start' && <StartScreen />}  
  
      {gameState === 'playing' && (  

```

```

<div
  className="game-area"
  style={{
    ...backgroundStyle,
    backgroundSize: 'cover',
    backgroundPosition: 'center',
    backgroundRepeat: 'no-repeat'
  }}
  ref={gameAreaRef}
>
  {renderThemeEffect()}

  {settings.showHUD && (
    <div className="hud">
      <div className="score">Score: {scoreState.score}</div>

      <div className="timer">
        <div className="timer-bar">
          <div
            className={`timer-fill ${
              timeLeft > settings.gameDuration * 0.5
                ? "green"
                : timeLeft > settings.gameDuration * 0.2
                ? "yellow"
                : "red"
            }`}
            style={{ width: `${(timeLeft / settings.gameDuration) * 100}%` }}
          ></div>
        </div>
      </div>
    )}
  )}

```



```

    <span
      className={
        timeLeft > settings.gameDuration * 0.5
          ? "green-text"
        : timeLeft > settings.gameDuration * 0.2
          ? "yellow-text"
        : "red-text"
      }
    >
      {timeLeft}s
    </span>
  </div>

  <div className="combo">
    {scoreState.combo > 0 && `Combo: ${scoreState.combo}x`}
  </div>
</div>
)}

{circles.map(circle => (
  <Circle
    key={circle.id}
    circle={circle}
    onCircleClick={handleCircleClick}
    targetSize={settings.targetSize}
  />
))}

{floatingScores.map(floatingScore => (

```

```

<FloatingScore
  key={floatingScore.id}
  x={floatingScore.x}
  y={floatingScore.y}
  points={floatingScore.points}
  onRemove={() => removeFloatingScore(floatingScore.id)}
/>
)}}

```

```

{particles.map(particle => (
  <div
    key={particle.id}
    className="particle"
    style={{
      left: particle.x,
      top: particle.y,
      backgroundColor: particle.color,
      transform: `rotate(${particle.angle}deg)`
    }}
  />
)}}
</div>
)}

```

```

{gameState === 'ended' && (
  <EndScreen
    scoreState={scoreState}
    happinessPercentage={happinessPercentage}
    settings={settings}

```

```

    />
  })
</div>
</GameContext.Provider>

);
};

// Start Screen Component
const StartScreen = () => {
  const { settings, dispatchSettings, startGame } = useContext(GameContext);

  return (
    <div className="start-screen">
      <div className="start-container">
        <h1 className="game-title">Find Your Day</h1>
        <p className="game-subtitle">Catch the emojis that match your mood!</p>

        <div className="settings-grid">
          <div className="setting-group">
            <label>Difficulty</label>
            <div className="button-group">
              {[ 'easy', 'medium', 'hard' ].map(diff => (
                <button
                  key={diff}
                  className={settings.difficulty === diff ? 'active' : ''}
                  onClick={() => dispatchSettings({ type: 'SET_DIFFICULTY', payload: diff })}
                >
                  {diff.charAt(0).toUpperCase() + diff.slice(1)}
                </button>
              ))}
            </div>
          </div>
        </div>
      </div>
    </div>
  );
};

```

```
    )}}  
  </div>  
</div>
```

```
<div className="setting-group">  
  <label>Target Size</label>  
  <div className="button-group">  
    {[ 'small', 'medium', 'large' ].map(size => (  
      <button  
        key={size}  
        className={settings.targetSize === size ? 'active' : ''}  
        onClick={() => dispatchSettings({ type: 'SET_SIZE', payload: size })}  
      >  
        {size.charAt(0).toUpperCase() + size.slice(1)}  
      </button>  
    )}}  
  </div>  
</div>
```

```
<div className="setting-group">  
  <label>Duration</label>  
  <div className="button-group">  
    {[30, 60, 90].map(duration => (  
      <button  
        key={duration}  
        className={settings.gameDuration === duration ? 'active' : ''}  
        onClick={() => dispatchSettings({ type: 'SET_DURATION', payload: duration })}  
      >  
        {duration}s  
      </button>  
    )}}  
  </div>  
</div>
```

```
    </button>
  )}
</div>
</div>
```

```
<div className="setting-group">
  <label>Mood Focus</label>
  <div className="button-group">
    {[ 'happy', 'sad', 'angry', 'surprise' ].map(category => (
      <button
        key={category}
        className={settings.categoryFocus === category ? 'active' : ''}
        onClick={() => dispatchSettings({ type: 'SET_CATEGORY', payload: category })}
      >
        {category.charAt(0).toUpperCase() + category.slice(1)}
      </button>
    ))}
  </div>
</div>
```

```
<div className="setting-group">
  <label>Theme Effect</label>
  <div className="button-group">
    {[
      { value: 'none', label: 'None' },
      { value: 'snow', label: 'Snow ❄️' },
      { value: 'wind', label: 'Wind 🌿' },
      { value: 'butterfly', label: 'Butterfly 🦋' },
      { value: 'romantic', label: 'Romantic 💕' }
    ]}
```

```

    ].map(theme => (
      <button
        key={theme.value}
        className={settings.themeEffect === theme.value ? 'active' : ''}
        onClick={() => dispatchSettings({ type: 'SET_THEME', payload: theme.value })}
      >
        {theme.label}
      </button>
    ))
  </div>
</div>

<div className="setting-group">
  <label className="toggle-label">
    <input
      type="checkbox"
      checked={settings.showHUD}
      onChange={() => dispatchSettings({ type: 'TOGGLE_HUD' })}
    />
    Show HUD
  </label>
</div>
</div>

<button className="start-button pulse" onClick={startGame}>
  <span>Start Game</span>
</button>
</div>
</div>

```

```
);  
};
```

```
const EndScreen = ({ scoreState, happinessPercentage }) => {  
  const { resetGame, dispatchSettings } = useContext(GameContext);  
  
  const getFeedbackMessage = (percentage) => {  
    if (percentage >= 80) return "You're radiating pure happiness! 🌞";  
    if (percentage >= 60) return "Great mood balance! Keep it up! 😊";  
    if (percentage >= 40) return "Mixed emotions today - that's perfectly normal! 😏";  
    if (percentage >= 20) return "Seems like a challenging day. Take care of yourself! 💙";  
    return "Everyone has tough days. You're stronger than you know! 💪";  
  };  
  
  const handleShare = async (platform) => {  
    const text = `🎮 Game Over!  
I just scored ${scoreState.score} points in Find Your Day!  
Hits: ${scoreState.hits}, Misses: ${scoreState.misses}  
Happiness: ${happinessPercentage}%`;  
    try {  
      const brightPanel = document.querySelector(".end-screen") || document.body;  
      if (!brightPanel) throw new Error("No game result panel found to capture.");  
      const originalOpacity = brightPanel.style.opacity;  
      const originalBackground = brightPanel.style.backgroundColor;  
      brightPanel.style.opacity = "1";  
      brightPanel.style.backgroundColor = "#111";  
      await new Promise((res) => setTimeout(res, 100));  
      const canvas = await html2canvas(brightPanel, {  
        backgroundColor: "#111",  
        useCORS: true,
```

```

    scale: 2,
  });
  brightPanel.style.opacity = originalOpacity;
  brightPanel.style.backgroundColor = originalBackground;
  const dataUrl = canvas.toDataURL("image/png");
  const blob = await (await fetch(dataUrl)).blob();
  const file = new File([blob], "game_result.png", { type: "image/png" });
  if (navigator.canShare && navigator.canShare({ files: [file] })) {
    await navigator.share({ files: [file], text: "Check out my game results!" });
  } else {
    const link = document.createElement("a");
    link.href = dataUrl;
    link.download = "game_result.png";
    link.click();
    if (platform === "twitter") {
      window.open(`https://twitter.com/intent/tweet?text=${encodeURIComponent(text)}`,
        "_blank");
    } else if (platform === "whatsapp") {
      window.open(`https://wa.me/?text=${encodeURIComponent(text)}`, "_blank");
    } else if (platform === "facebook") {
      window.open(`https://www.facebook.com/sharer/sharer.php?u=${encodeURIComponent(
        text)}`, "_blank");
    } else if (platform === "telegram") {
      window.open(`https://t.me/share/url?url=${encodeURIComponent(text)}`, "_blank");
    } else if (platform === "instagram") {
      const formData = new FormData();
      formData.append("file", blob);
      formData.append("upload_preset", "kctvcgar");
      formData.append("folder", "game_result");
      try {

```



```

    const res = await fetch("https://api.cloudinary.com/v1_1/dw2zstp9c/upload", {
method: "POST", body: formData });

    const data = await res.json();

    const imageUrl = data.secure_url;

    if (imageUrl) {

        await navigator.clipboard.writeText(`Check my game result 🎮: ${imageUrl}`);

        alert("Instagram doesn't support direct sharing. ✅ Link copied! Paste it in DM or
story.");

    } else {

        alert("Upload succeeded but no URL returned.");

    }

} catch (err) {

    console.error("Cloudinary upload failed:", err);

    alert("Failed to upload image for Instagram sharing.");

}

} else {

    await navigator.clipboard.writeText(text);

    alert(`Game stats copied to clipboard! Share them on ${platform}.`);

}

}

} catch (error) {

    console.error("Error during sharing process:", error);

    alert("Failed to share game results. Please try again.");

}

};

return (

    <div className="end-screen">

```

```
<div className="end-container">

  <h2 className="end-title">Game Over .!!</h2>

  { /* Score Grid */ }

  <div className="stats-grid">

    <div className="stat-card">

      <div className="stat-number">{scoreState.score}</div>

      <div className="stat-label">Total Score</div>

    </div>

    <div className="stat-card">

      <div className="stat-number">{scoreState.hits}</div>

      <div className="stat-label">Hits</div>

    </div>

    <div className="stat-card">

      <div className="stat-number">{scoreState.misses}</div>

      <div className="stat-label">Misses</div>

    </div>

    <div className="stat-card">

      <div className="stat-number">{happinessPercentage}%</div>

      <div className="stat-label">Happiness</div>

    </div>

  </div>

  { /* Happiness Meter */ }

  <div className="happiness-meter">

    <div className="happiness-bar">

      <div

        className="happiness-fill"

        style={{ width: `${happinessPercentage}%` }}

      >
```

```

    ></div>

</div>

<p className="feedback-message">{getFeedbackMessage(happinessPercentage)}</p>
</div>

{/* Category Breakdown */}
<div className="category-breakdown">
  <h3>Category Breakdown</h3>
  <div className="category-bars">
    {Object.entries(scoreState.categoryScores).map(([category, score]) => (
      <div key={category} className="category-bar">
        <span className="category-name">
          {category.charAt(0).toUpperCase() + category.slice(1)}
        </span>
        <div className="bar-container">
          <div
            className={`bar ${category}`}
            style={{
              width: `${
                (score / Math.max(...Object.values(scoreState.categoryScores))) *
                100
              }%`,
            }}
          ></div>
          </div>
          <span className="category-score">{score}</span>
        </div>
      )
    )}
  </div>

```

```
</div>
```

```
{/* Buttons */}
```

```
<div className="action-buttons">
```

```
  <button className="action-button primary" onClick={resetGame}>
```

```
    Play Again
```

```
  </button>
```

```
  <button
```

```
    className="action-button secondary"
```

```
    onClick={() =>
```

```
      dispatchSettings({
```

```
        type: "RESET",
```

```
        payload: {
```

```
          difficulty: "medium",
```

```
          targetSize: "medium",
```

```
          gameDuration: 60,
```

```
          categoryFocus: "happy",
```

```
          themeEffect: "none",
```

```
          showHUD: true,
```

```
        },
```

```
      })
```

```
    }
```

```
  >
```

```
    Change Settings
```

```
  </button>
```

```
</div>
```

```
{/* Social Icons */}
```

```
<div className="share-icons">
```

```
<button className="icon-btn twitter" onClick={() => handleShare("twitter")}>
  <FaTwitter />
</button>

<button className="icon-btn whatsapp" onClick={() => handleShare("whatsapp")}>
  <FaWhatsapp />
</button>

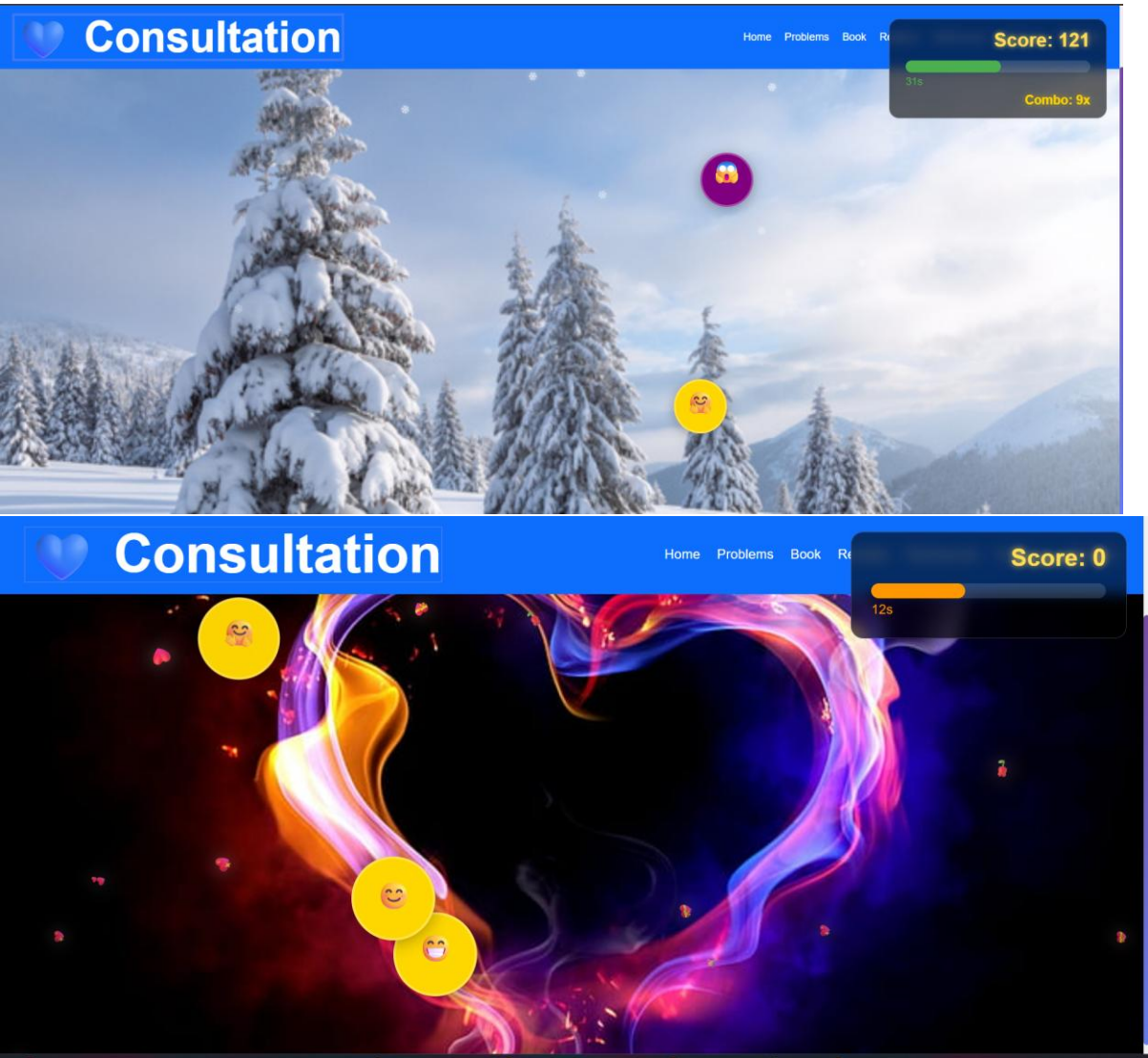
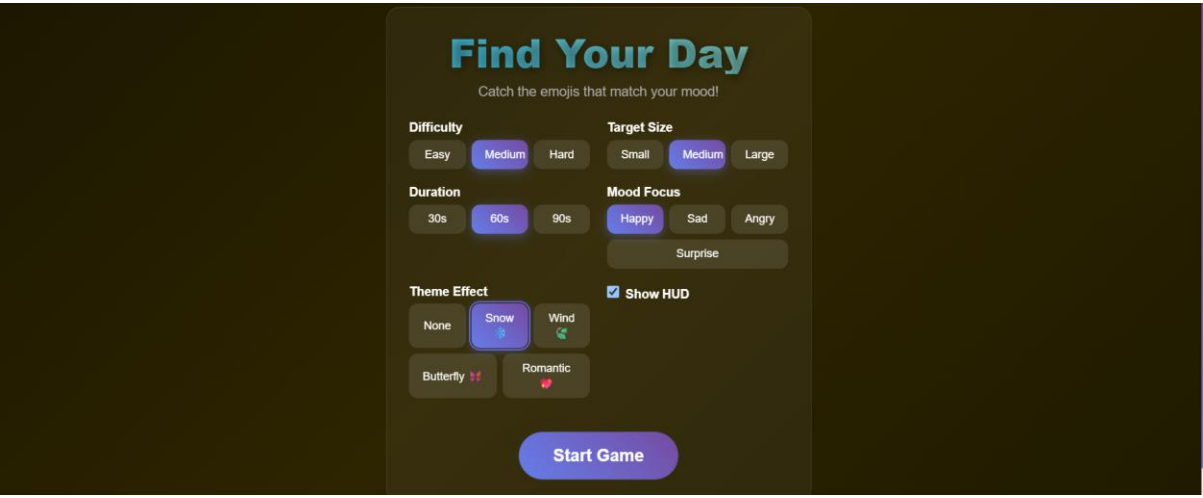
<button className="icon-btn instagram" onClick={() => handleShare("instagram")}>
  <FaInstagram />
</button>

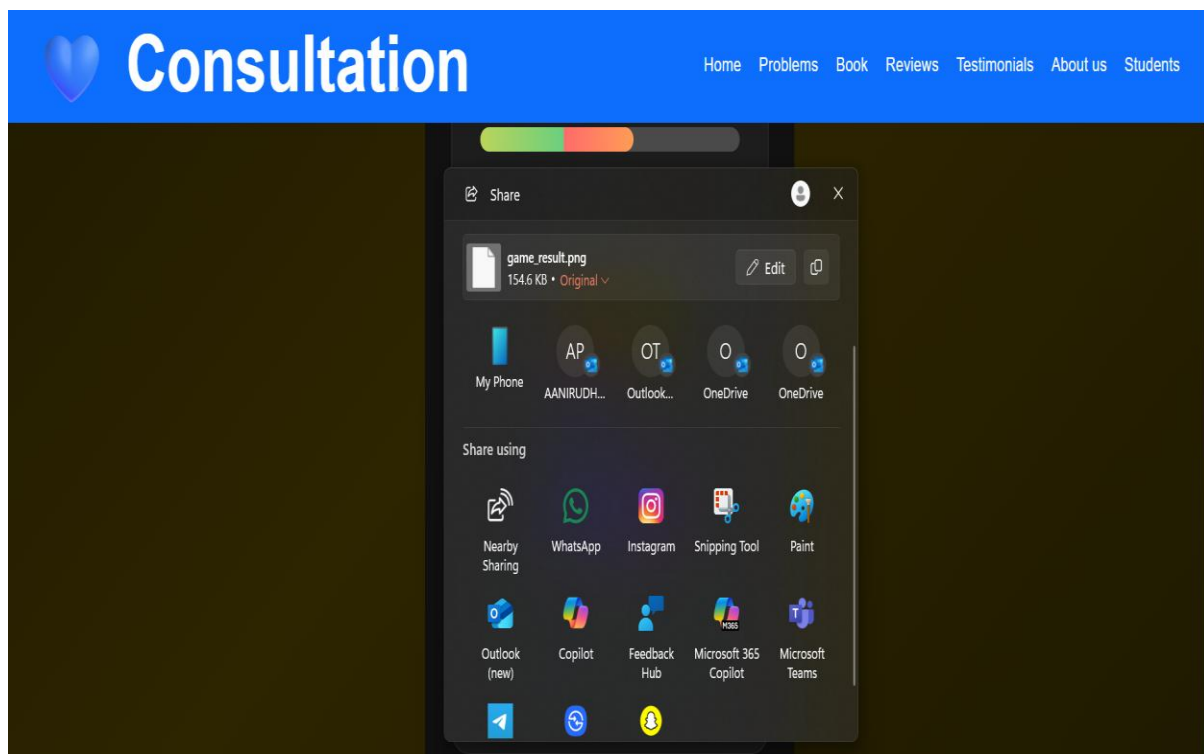
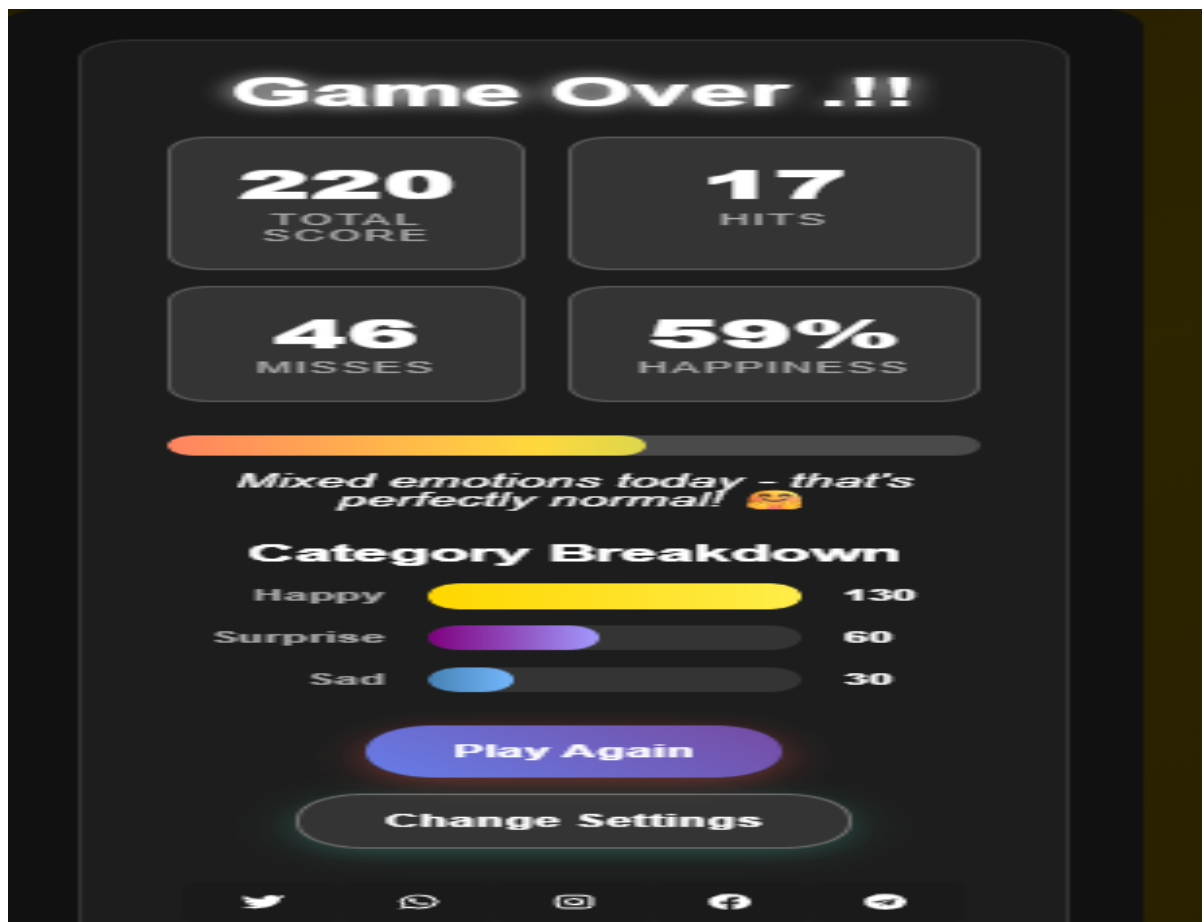
<button className="icon-btn facebook" onClick={() => handleShare("facebook")}>
  <FaFacebook />
</button>

<button className="icon-btn telegram" onClick={() => handleShare("telegram")}>
  <FaTelegram />
</button>
</div>
</div>
</div>
);
};

export default Find;
```

OUTPUT:-





---

Ex No:10

# REACT RENDERING

Date:01.08.2025

---

## **AIM:-**

Develop a webpage to demonstrate the following using React JS:

Basic Rendering (Element Rendering)

Conditional Rendering

Component Rendering

List Rendering - use key & map()

Inline CSS rendering

Logical && AND Rendering

## **DESCRIPTION:-**

Basic Rendering – Simple <p> and <h1> elements.

Conditional Rendering – isLoggedIn ? ... : ....

Component Rendering – ChildComponent rendered inside App.

List Rendering – fruits.map() with key.

Inline CSS – style={inlineStyle} applied to a <div>.

Logical AND (&&) Rendering – Show message only if isLoggedIn is true.



## **CODE:-**

```
import React, { useState } from "react";

import './App.css';

// Child Component for Component Rendering
const ChildComponent = ({ message }) => {
  return <h3 style={{ color: "blue" }}>{message}</h3>;
};

function App() {
  // Conditional Rendering
  const [isLoggedIn, setIsLoggedIn] = useState(false);

  // List Rendering
  const fruits = ["Apple", "Banana", "Cherry", "Date"];

  // Inline CSS
  const inlineStyle = {
    backgroundColor: "#f0f0f0",
    padding: "10px",
    borderRadius: "5px",
    margin: "10px 0"
  };

  return (
    <div className="App">
      <h1>React Rendering Demo</h1>

      { /* 1. Basic Rendering */ }
```

## <h2> **1** Basic Rendering</h2>

<p>This is a basic element rendered using React JSX.</p>

{/\* 2. Conditional Rendering \*/}

## <h2> **2** Conditional Rendering</h2>

{isLoggedIn ? (

<p>Welcome, User! You are logged in.</p>

) : (

<p>Please log in to continue.</p>

)}</div>

<button onClick={() => setIsLoggedIn(!isLoggedIn)}>

{isLoggedIn ? "Logout" : "Login"}</div>

</button>

{/\* 3. Component Rendering \*/}

## <h2> **3** Component Rendering</h2>

<ChildComponent message="This is a child component!" />

{/\* 4. List Rendering \*/}

## <h2> **4** List Rendering</h2>

<ul>

{fruits.map((fruit, index) => (

<li key={index}>{fruit}</li>

))}

</ul>

{/\* 5. Inline CSS Rendering \*/}

## <h2> **5** Inline CSS Rendering</h2>

<div style={inlineStyle}>This div is styled using inline CSS!</div>

```
{/* 6. Logical && AND Rendering */}
```

```
<h2> 6 Logical && AND Rendering</h2>
```

```
{isLoggedIn && <p>You can see this message because you are logged in.</p>}
```

```
</div>
```

```
);
```

```
}
```

```
export default App;
```

## OUTPUT:-

# React Rendering Demo

## 1 Basic Rendering

This is a basic element rendered using React JSX.

## 2 Conditional Rendering

Please log in to continue.

Login

## 3 Component Rendering

**This is a child component!**

## 4 List Rendering

- Apple
- Banana
- Cherry
- Date

## 5 Inline CSS Rendering

This div is styled using inline CSS!

## 6 Logical && AND Rendering

## 2 Conditional Rendering

Welcome, User! You are logged in.

Logout

## 2 Conditional Rendering

Please log in to continue.

Login

## **AIM:-**

Develop a webpage to demonstrate the following using React JS:

❑ Implement React Router to create a menu layout page and

link to various pages using the menu.

❑ Design React Forms with maximum variety of fields and

perform appropriate validations.

## **DESCRIPTION:-**

How my navbar.jsx helps

- Navigation Links with Link
- Each `<Link to="...">` element corresponds to a route in your app.
- Clicking a link updates the URL without refreshing the page, which is the core feature of React Router.
- Centralized Menu Layout
- `Navbar.jsx` serves as a single, reusable navigation menu component.
- You can include it in your main layout so every page shares the same navigation structure.
- Integration with React Router Routes
- These links work in combination with `<Route>` components (defined in your main router file) to render the correct page/component when a URL is visited.
- Example: `Link to="/consultation"` triggers the Consultation component defined in your routes.
- Improves Component Reusability and Readability
- By separating the navbar into its own component, your code is cleaner and easier to maintain. Any future changes to the navigation (like adding new pages) require changes only in `Navbar.jsx`, not in every page individually.

For forms

The Register Form in your Register.jsx is implemented as a multi-step registration form with React state management. Here's how it works:

#### Step-wise Form Navigation

Registration is split into Step 1 and Step 2.

Step 1 collects personal details: name, email, phone, DOB, address, extra fields, and optional profile photo.

Step 2 collects account credentials: username, password, and confirm password.

Navigation between steps is handled by registerStep state.

#### Controlled Inputs with Validation

Each input is a controlled component: its value is stored in registerData state.

Required fields use the required attribute, and password confirmation is validated before submission.

#### Form Submission

Step 1 → calls handleRegisterStep1 → advances to step 2.

Step 2 → calls handleRegisterStep2 → sends data to backend as FormData (to include optional file upload).

If successful, a success message is shown, and the form resets.

#### Optional File Upload

Users can upload a profile photo. The file is appended to FormData and sent to the backend.

#### UI Feedback & Navigation

Success and error messages are shown dynamically.

Users can go back to login anytime with the Back to Login button.

## **CODE:-**

Navbar.jsx

```
import React from "react";  
import { Link } from "react-router-dom";
```

```
import './Navbar.css';

function Navbar() {
  return (
    <nav className="navbar">
      <div className="navbar-container">
        <h2 className="logo">  Consultation</h2>
        <div className="nav-links">
          <Link to="/Find">Home</Link>

          <Link to="/problems">Problems</Link>
          <Link to="/consultation">Book</Link>
          <Link to="/reviews">Reviews</Link>
          <Link to="/testimonials">Testimonials</Link>
          <Link to="/Sad">About us</Link>

          <Link to="/students">Students</Link>
        </div>
      </div>
    </nav>
  );
}

export default Navbar;
```

Register.jsx form-



```

import React, { useState } from "react";

import { User, Mail, Phone, Calendar, MapPin, FileText, Lock, Eye, EyeOff } from "lucide-react";

const Register = ({ switchView }) => {
  const [registerStep, setRegisterStep] = useState(1);
  const [showPassword, setShowPassword] = useState(false);
  const [showConfirmPassword, setShowConfirmPassword] = useState(false);
  const [registerData, setRegisterData] = useState({
    name: "", email: "", phone: "", dob: "", address: "", extra1: "", extra2: "",
    username: "", password: "", confirmPassword: "", profilePhoto: null
  });

  const handleStep1 = (e) => { e.preventDefault(); setRegisterStep(2); };
  const handleStep2 = (e) => {
    e.preventDefault();
    if(registerData.password !== registerData.confirmPassword){ alert("Passwords do not match"); return; }
    console.log(registerData);
    alert("Registered successfully!");
    switchView("login");
  };

  return (
    <div className="register-card">
      {registerStep === 1 && (
        <form onSubmit={handleStep1}>
          <h2>Step 1: Personal Info</h2>
          <input type="text" placeholder="Full Name" required

```

```

        value={registerData.name} onChange={e=>setRegisterData({...registerData,
name:e.target.value})}/>

        <input type="email" placeholder="Email" required

        value={registerData.email} onChange={e=>setRegisterData({...registerData,
email:e.target.value})}/>

        <input type="tel" placeholder="Phone" required

        value={registerData.phone} onChange={e=>setRegisterData({...registerData,
phone:e.target.value})}/>

        <input type="date" placeholder="DOB" required

        value={registerData.dob} onChange={e=>setRegisterData({...registerData,
dob:e.target.value})}/>

        <input type="text" placeholder="Address (Optional)"

        value={registerData.address} onChange={e=>setRegisterData({...registerData,
address:e.target.value})}/>

        <input type="file" onChange={e=>setRegisterData({...registerData,
profilePhoto:e.target.files[0]})}/>

        <button type="submit">Next Step</button>

    </form>

)}

{registerStep === 2 && (
    <form onSubmit={handleStep2}>
        <h2>Step 2: Account Info</h2>
        <input type="text" placeholder="Username" required

        value={registerData.username} onChange={e=>setRegisterData({...registerData,
username:e.target.value})}/>

        <input type={showPassword?"text":"password"} placeholder="Password" required

        value={registerData.password} onChange={e=>setRegisterData({...registerData,
password:e.target.value})}/>

        <input type={showConfirmPassword?"text":"password"} placeholder="Confirm
Password" required

        value={registerData.confirmPassword}
onChange={e=>setRegisterData({...registerData, confirmPassword:e.target.value})}/>

```

```
<button type="submit">Create Account</button>

<button type="button" onClick={()=>setRegisterStep(1)}>Previous Step</button>

<button type="button" onClick={()=>switchView("login")}>Back to Login</button>

</form>

  })
</div>

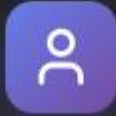
);

};

export default Register;
```

## **OUTPUT:-**





# Welcome

## Create Account - Step 1

 Gajendra

 gajendra\_cpm@gmail.com

 901018902

 20-02-2005



 Address (Optional)

 I got to know through my friends

 I want to get clarity of mind

Profile Photo (Optional):

Choose File gajendra.jpeg

Next Step

Back to Login



Welcome

Create Account - Step 2

 Gajendra

 .... 

 .... 

Create Account

Previous Step

Back to Login



Welcome

Create Account - Step 2

 Username

 Password 

 Confirm Password 

Create Account

Previous Step

Back to Login

**AIM:-**

Use NodeJS to Connect to a database and perform CRUD operations.

Create Database, tables to store and retrieve relevant values.

**CODE:-**

```
/* eslint-disable no-unused-vars */  
  
// student_management.js  
  
import mysql from 'mysql2';  
  
  
// 1. Connect to MySQL server  
const con = mysql.createConnection({  
  host: "localhost",  
  user: "root",  
  password: "aanirudh_02"  
});  
  
  
// 2. Connect to MySQL  
con.connect((err) => {  
  if (err) throw err;  
  console.log("Connected to MySQL server!");  
  
  
// 3. Create Database  
con.query("CREATE DATABASE IF NOT EXISTS StudentDB", (err, result) => {  
  if (err) throw err;  
  console.log("Database created or already exists!");  
});
```

```
// Use the database

con.query("USE StudentDB", (err, result) => {
  if (err) throw err;

  // 4. Create Table
  const createTable = `
    CREATE TABLE IF NOT EXISTS Students (
      id INT AUTO_INCREMENT PRIMARY KEY,
      name VARCHAR(50) NOT NULL,
      age INT NOT NULL,
      email VARCHAR(50) UNIQUE NOT NULL,
      course VARCHAR(50)
    )
  `;
  con.query(createTable, (err, result) => {
    if (err) throw err;
    console.log("Students table created or already exists!");

    // 5. Insert Sample Students
    const insertStudents = `
      INSERT INTO Students (name, age, email, course)
      VALUES
        ('Alice', 20, 'alice@example.com', 'Math'),
        ('Bob', 22, 'bob@example.com', 'Physics'),
        ('Charlie', 21, 'charlie@example.com', 'Chemistry')
      ON DUPLICATE KEY UPDATE name=name
    `;
    con.query(insertStudents, (err, result) => {
```

```
if (err) throw err;

console.log("Sample students inserted!");


// 6. Read all students

con.query("SELECT * FROM Students", (err, results) => {
  if (err) throw err;
  console.log("All Students:");
  console.table(results);

// 7. Update a student

const updateQuery = "UPDATE Students SET course='Biology' WHERE name='Alice'";
con.query(updateQuery, (err, result) => {
  if (err) throw err;
  console.log("Updated Alice's course!");

// 8. Delete a student

const deleteQuery = "DELETE FROM Students WHERE name='Bob'";
con.query(deleteQuery, (err, result) => {
  if (err) throw err;
  console.log("Deleted Bob from students!");

// 9. Final list

con.query("SELECT * FROM Students", (err, results) => {
  if (err) throw err;
  console.log("Final Students List:");
  console.table(results);

// Close the connection

con.end();
```



```
    });  
  });  
});  
});  
});  
});  
});  
});  
});
```

## OUTPUT:-

```
E:\React_Projects\consultation>node server/new.js  
Connected to MySQL server!  
Database created or already exists!  
Students table created or already exists!  
Sample students inserted!  
All Students:
```

| (index) | id | name      | age | email                 | course      |
|---------|----|-----------|-----|-----------------------|-------------|
| 0       | 1  | 'Alice'   | 20  | 'alice@example.com'   | 'Math'      |
| 1       | 2  | 'Bob'     | 22  | 'bob@example.com'     | 'Physics'   |
| 2       | 3  | 'Charlie' | 21  | 'charlie@example.com' | 'Chemistry' |

```
Updated Alice's course!  
Deleted Bob from students!  
Final Students List:
```

| (index) | id | name      | age | email                 | course      |
|---------|----|-----------|-----|-----------------------|-------------|
| 0       | 1  | 'Alice'   | 20  | 'alice@example.com'   | 'Biology'   |
| 1       | 3  | 'Charlie' | 21  | 'charlie@example.com' | 'Chemistry' |

```
E:\React_Projects\consultation>
```

**AIM:-**

Develop a complete website using sign-in page, and other relevant pages(components).

Connect the frontend Client (ReactJs) with backend server (NodeJs) and Database (MySQL)

Use Axios and ExpressJS.

**CODE:-**

-

```
/* eslint-disable no-unused-vars */
```

```
import React, { useState } from 'react';
```

```
import { Eye, EyeOff, User, Lock, Mail, Phone, Calendar, MapPin, FileText } from 'lucide-react';
```

```
import './Login.css';
```

```
import { useNavigate } from "react-router-dom";
```

```
const guestNamePrefixes = ['anonymous_', 'anonymous', 'anon_', 'anon', 'Guest', 'guest_',  
'guest', 'g_user', 'guest'];
```

```
const randomChar = () => {
```

```
  const chars =  
'0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz';
```

```
  return chars.charAt(Math.floor(Math.random() * chars.length));
```

```
};
```

```
const generateGuestUsername = () => {
```

```
  const prefix = guestNamePrefixes[Math.floor(Math.random() * guestNamePrefixes.length)];
```

```
  let suffix;
```

```
  if (prefix.match(/[A-Za-z_]$/)) {
```

```

    suffix = (Math.floor(Math.random() * 99) + 1).toString().padStart(2, '0');
  } else if (prefix.match(/^\D+$/)) {
    suffix = randomChar();
  } else {
    suffix = (Math.floor(Math.random() * 99) + 1).toString().padStart(2, '0');
  }
  return prefix + suffix;
};

```

```

const Login = () => {
  const [currentView, setCurrentView] = useState('login');
  const [showPassword, setShowPassword] = useState(false);
  const [showConfirmPassword, setShowConfirmPassword] = useState(false);
  const [showGuestPassword, setShowGuestPassword] = useState(false);
  const [showNewPassword, setShowNewPassword] = useState(false);
  const [showConfirmNewPassword, setShowConfirmNewPassword] = useState(false);
  const [registerStep, setRegisterStep] = useState(1);
  const [guestCredentials, setGuestCredentials] = useState(null);
  const [showSuccessMessage, setShowSuccessMessage] = useState(false);
  const [successMessage, setSuccessMessage] = useState('');
  const [showPasswordChangeForm, setShowPasswordChangeForm] = useState(false);
  const [guestPasswordChanged, setGuestPasswordChanged] = useState(false);

  const [loginData, setLoginData] = useState({
    username: '',
    password: ''
  });

```

```

const [registerData, setRegisterData] = useState({

```

```
name: "",
email: "",
phone: "",
dob: "",
address: "",
extra1: "",
extra2: "",
profilePhoto: null, // file object
username: "",
password: "",
confirmPassword: ""
});
```

```
const [guestData, setGuestData] = useState({
  username: "",
  password: ""
});
```

```
const [passwordChangeData, setPasswordChangeData] = useState({
  newPassword: "",
  confirmNewPassword: ""
});
```

```
// Backend base URL
```

```
const API_BASE = "http://localhost:3001/api";
```

```
const navigate = useNavigate();
```

```
const handleLogin = async (e) => {
```

```
e.preventDefault();

try {

  const res = await fetch(`${API_BASE}/login`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(loginData),
  });

  const data = await res.json();

  if (res.ok) {

    setSuccessMessage(`Login successful! Welcome, ${data.user.username || data.user.name}`);

    setShowSuccessMessage(true);

    // redirect after short delay
    setTimeout(() => {
      if (data.userType === "anonymous") {
        // 👉 pass username into state
        navigate("/anonymous", {
          state: { guestCredentials: { username: data.user.username || "AnonymousUser" } }
        });
      } else {
        navigate("/find", {
          state: { userDetails: data.user } // 👉 pass normal user data
        });
      }
    }, 1500);
  } else {
    alert(data.error || "Login failed");
  }
}
```

```
    }  
  } catch (error) {  
    alert("Network error. Please try again.");  
    console.error(error);  
  }  
};
```

```
// Register handlers
```

```
const handleRegisterStep1 = (e) => {  
  e.preventDefault();  
  setRegisterStep(2);  
};
```

```
const handleRegisterStep2 = async (e) => {  
  e.preventDefault();  
  if (registerData.password !== registerData.confirmPassword) {  
    alert('Passwords do not match!');  
    return;  
  }  
}
```

```
// Prepare FormData to handle file upload if profilePhoto is a file
```

```
const formData = new FormData();  
formData.append('name', registerData.name);  
formData.append('email', registerData.email);  
formData.append('phone', registerData.phone);  
formData.append('dob', registerData.dob);  
formData.append('address', registerData.address);  
formData.append('extra1', registerData.extra1);  
formData.append('extra2', registerData.extra2);
```

```

formData.append('username', registerData.username);
formData.append('password', registerData.password);
if (registerData.profilePhoto) {
  formData.append('profilePhoto', registerData.profilePhoto);
}

try {
  const res = await fetch(`${API_BASE}/register`, {
    method: "POST",
    body: formData // sending as multipart/form-data
  });
  const data = await res.json();
  if (!res.ok) {
    alert(data.error || "Registration failed");
    return;
  }
  setSuccessMessage('Registration successful! Redirecting to login...');
  setShowSuccessMessage(true);
  setTimeout(() => {
    setShowSuccessMessage(false);
    setCurrentView('login');
    setRegisterStep(1);
    setRegisterData({
      name: "", email: "", phone: "", dob: "", address: "", extra1: "", extra2: "", profilePhoto: null,
      username: "", password: "", confirmPassword: ""
    });
    resetPasswordStates();
  }, 2000);
} catch (error) {

```

```

    alert("Could not complete registration. Network error.");
    console.error(error);
  }
};

// Generate guest and save to backend
const generateGuestCredentials = async () => {
  const randomUsername = generateGuestUsername();
  const randomPassword = Math.random().toString(36).substr(2, 8);
  const credentials = {
    username: randomUsername,
    password: randomPassword
  };

  try {
    const res = await fetch(`${API_BASE}/anonymous`, {
      method: "POST",
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(credentials),
    });
    if (!res.ok) {
      console.error("Failed to save anonymous user");
    }
  } catch (error) {
    console.error("Error saving anonymous user", error);
  }

  setGuestCredentials(credentials);
  setGuestData(credentials);

```



```
    setGuestPasswordChanged(false);  
    return credentials;  
};
```

```
const handleContinueAsGuest = () => {  
    generateGuestCredentials();  
    setCurrentView('guest');  
};
```

```
const handlePasswordChangeSubmit = async (e) => {  
    e.preventDefault();  
    if (passwordChangeData.newPassword !== passwordChangeData.confirmNewPassword) {  
        alert('Passwords do not match!');  
        return;  
    }  
}
```

```
const updatedCredentials = {  
    ...guestCredentials,  
    password: passwordChangeData.newPassword,  
};  
setGuestCredentials(updatedCredentials);  
setGuestData(updatedCredentials);
```

```
//  Show confirmation screen  
setGuestPasswordChanged(true);
```

```
//  Keep the form container visible so confirmation replaces it  
setPasswordChangeData({ newPassword: "", confirmNewPassword: "" });
```

```

    resetPasswordStates();
};

const handleContinueAfterGuestPasswordChange = async () => {
  try {
    // send to the SAME endpoint you already wrote in backend
    await fetch("http://localhost:3001/api/anonymous", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(guestCredentials), // 👉 includes updated password
    });

    setSuccessMessage("Password updated successfully! Redirecting...");
    setShowSuccessMessage(true);

    setTimeout(() => {
      setShowSuccessMessage(false);
      setCurrentView("login");
      setGuestCredentials(null);
      setGuestData({ username: "", password: "" });
      setShowPasswordChangeForm(false);
      setGuestPasswordChanged(false);
      resetPasswordStates();
    }, 2000);
  } catch (err) {
    console.error("Failed to save updated guest password", err);
    alert("Could not update password. Try again.");
  }
};

```

```
const handleSuccessOk = () => {  
  setShowSuccessMessage(false);  
  if (currentView === 'guest' || successMessage.includes('Password changed')) {  
    setCurrentView('login');  
    setGuestCredentials(null);  
    setGuestData({ username: '', password: '' });  
    setShowPasswordChangeForm(false);  
    setPasswordChangeData({ newPassword: '', confirmNewPassword: '' });  
    resetPasswordStates();  
  }  
};
```

```
const resetPasswordStates = () => {  
  setShowPassword(false);  
  setShowConfirmPassword(false);  
  setShowGuestPassword(false);  
  setShowNewPassword(false);  
  setShowConfirmNewPassword(false);  
};
```

```
const switchView = (view) => {  
  setCurrentView(view);  
  setRegisterStep(1);  
  setShowPasswordChangeForm(false);  
  resetPasswordStates();  
  setGuestPasswordChanged(false);  
};
```

```

return (
  <div className="login-container">
    {showSuccessMessage && (
      <div className="success-overlay">
        <div className="success-modal">
          <div className="success-icon">✓</div>
          <p>{successMessage}</p>
          <button onClick={handleSuccessOk} className="ok-button">OK</button>
        </div>
      </div>
    )}
  </div>
)

```

```

<div className={`login-card ${currentView}`}>
  <div className="header-image">
    <div className="logo-placeholder">
      <User size={48} />
    </div>
    <h1 className="app-title">Welcome</h1>
  </div>

```

```

{/* Login Form */}
{currentView === 'login' && (
  <div className="form-section active">
    <h2>Sign In</h2>
    <form onSubmit={handleLogin}>
      <div className="input-group">
        <User className="input-icon" size={20} />
        <input
          type="text"

```

```
placeholder="Username"
value={loginData.username}
onChange={(e) => setLoginData({ ...loginData, username: e.target.value })}
autoComplete="off"
required
/>
</div>
```

```
<div className="input-group">
  <Lock className="input-icon" size={20} />
  <input
    type={showPassword ? 'text' : 'password'}
    placeholder="Password"
    value={loginData.password}
    onChange={(e) => setLoginData({ ...loginData, password: e.target.value })}
    autoComplete="off"
    required
  />
  <button
    type="button"
    className="password-toggle"
    onClick={() => setShowPassword(!showPassword)}
  >
    {showPassword ? <EyeOff size={20} /> : <Eye size={20} />}
  </button>
</div>

<button type="submit" className="primary-button">Sign In</button>
</form>
```

```

    <div className="form-footer">
      <button onClick={() => switchView('register')} className="link-button">Create
Account</button>
      <button onClick={handleContinueAsGuest} className="link-button">Continue as
Guest</button>
    </div>
  </div>
)}

```

```

{/* Register Form Step 1 */}
{currentView === 'register' && registerStep === 1 && (
  <div className="form-section active">
    <h2>Create Account - Step 1</h2>
    <form onSubmit={handleRegisterStep1}>
      <div className="input-group">
        <User className="input-icon" size={20} />
        <input
          type="text"
          placeholder="Full Name"
          value={registerData.name}
          onChange={(e) => setRegisterData({ ...registerData, name: e.target.value })}
          autoComplete="off"
          required
        />
      </div>
      <div className="input-group">
        <Mail className="input-icon" size={20} />
        <input

```

```
    type="email"
    placeholder="Email"
    value={registerData.email}
    onChange={(e) => setRegisterData({ ...registerData, email: e.target.value })}
    autoComplete="off"
    required
  />
</div>
```

```
<div className="input-group">
  <Phone className="input-icon" size={20} />
  <input
    type="tel"
    placeholder="Phone Number"
    value={registerData.phone}
    onChange={(e) => setRegisterData({ ...registerData, phone: e.target.value })}
    autoComplete="off"
    required
  />
</div>
```

```
<div className="input-group">
  <Calendar className="input-icon" size={20} />
  <input
    type="date"
    placeholder="Date of Birth"
    value={registerData.dob}
    onChange={(e) => setRegisterData({ ...registerData, dob: e.target.value })}
    autoComplete="off"
  />
</div>
```

```
      required
    />
  </div>
```

```
<div className="input-group">
  <MapPin className="input-icon" size={20} />
  <input
    type="text"
    placeholder="Address (Optional)"
    value={registerData.address}
    onChange={(e) => setRegisterData({ ...registerData, address: e.target.value })}
    autoComplete="off"
  />
</div>
```

```
<div className="input-group">
  <FileText className="input-icon" size={20} />
  <input
    type="text"
    placeholder="Extra Detail 1"
    value={registerData.extra1}
    onChange={(e) => setRegisterData({ ...registerData, extra1: e.target.value })}
    autoComplete="off"
  />
</div>
```

```
<div className="input-group">
  <FileText className="input-icon" size={20} />
  <input
```



```

        type="text"
        placeholder="Extra Detail 2"
        value={registerData.extra2}
        onChange={(e) => setRegisterData({ ...registerData, extra2: e.target.value })}
        autoComplete="off"
      />
    </div>

    { /* Profile photo optional input */ }
    <div className="input-group">
      <label htmlFor="profilePhoto">Profile Photo (Optional):</label>
      <input
        type="file"
        id="profilePhoto"
        accept="image/*"
        onChange={(e) => setRegisterData({ ...registerData, profilePhoto: e.target.files[0]
    }}}
      />
    </div>

    <button type="submit" className="primary-button">Next Step</button>
  </form>

  <div className="form-footer">
    <button onClick={() => switchView('login')} className="link-button">Back to
Login</button>
  </div>
</div>
)}

```

```

{/* Register Form Step 2 */}

{currentView === 'register' && registerStep === 2 && (
  <div className="form-section active">
    <h2>Create Account - Step 2</h2>
    <form onSubmit={handleRegisterStep2}>
      <div className="input-group">
        <User className="input-icon" size={20} />
        <input
          type="text"
          placeholder="Username"
          value={registerData.username}
          onChange={(e) => setRegisterData({ ...registerData, username: e.target.value })}
          autoComplete="off"
          required
        />
      </div>

      <div className="input-group">
        <Lock className="input-icon" size={20} />
        <input
          type={showPassword ? 'text' : 'password'}
          placeholder="Password"
          value={registerData.password}
          onChange={(e) => setRegisterData({ ...registerData, password: e.target.value })}
          autoComplete="off"
          required
        />
        <button
          type="button"

```

```

        className="password-toggle"
        onClick={() => setShowPassword(!showPassword)}
      >
        {showPassword ? <EyeOff size={20} /> : <Eye size={20} />}
      </button>
    </div>

    <div className="input-group">
      <Lock className="input-icon" size={20} />

      <input
        type={showConfirmPassword ? 'text' : 'password'}
        placeholder="Confirm Password"
        value={registerData.confirmPassword}
        onChange={(e) => setRegisterData({ ...registerData, confirmPassword:
e.target.value })}
        autoComplete="off"
        required
      />

      <button
        type="button"
        className="password-toggle"
        onClick={() => setShowConfirmPassword(!showConfirmPassword)}
      >
        {showConfirmPassword ? <EyeOff size={20} /> : <Eye size={20} />}
      </button>
    </div>

    <button type="submit" className="primary-button">Create Account</button>
  </form>

```

```

        <div className="form-footer">
            <button onClick={() => setRegisterStep(1)} className="link-button">Previous
Step</button>
            <button onClick={() => switchView('login')} className="link-button">Back to
Login</button>
        </div>
    </div>
)}

```

```

{/* Guest Form */}

```

```

{/* Guest Form */}

```

```

{currentView === 'guest' && guestCredentials && !showPasswordChangeForm &&
!guestPasswordChanged && (

```

```

    <div className="form-section active">

```

```

        <h2>Guest Access</h2>

```

```

        <div className="guest-credentials">

```

```

            <p className="guest-info">Your temporary credentials have been generated:</p>

```

```

        <div className="input-group readonly">

```

```

            <User className="input-icon" size={20} />

```

```

            <input type="text" value={guestCredentials.username} readOnly />

```

```

        </div>

```

```

        <div className="input-group readonly">

```

```

            <Lock className="input-icon" size={20} />

```

```

            <input

```

```

                type={showGuestPassword ? 'text' : 'password'}

```

```

                value={guestCredentials.password}

```

```

                readOnly

```

```

/>
<button
  type="button"
  className="password-toggle"
  onClick={() => setShowGuestPassword(!showGuestPassword)}
>
  {showGuestPassword ? <EyeOff size={20} /> : <Eye size={20} />}
</button>
</div>

```

```

<div className="guest-buttons">
  {/* Change password */}
  <button
    onClick={() => setShowPasswordChangeForm(true)}
    className="secondary-button"
  >
    Change Password
  </button>

```

```

<button
  onClick={() => {
    setSuccessMessage("Guest access successful! Redirecting to login...");
    setShowSuccessMessage(true);

    // Reset guest state (just like after password change / register)
    setGuestCredentials(null);
    setGuestData({ username: "", password: "" });
    setShowPasswordChangeForm(false);
    setGuestPasswordChanged(false);
  }}

```

```

// Automatically redirect after 2s
setTimeout(() => {
  setShowSuccessMessage(false);
  setCurrentView('login');
}, 2000);
}}
className="primary-button"
>
Continue
</button>

</div>
</div>

<div className="form-footer">
  <button onClick={() => switchView('login')} className="link-button">Back to
Login</button>
</div>
</div>
)}

```

```

{/* Password Change Form */}
{currentView === 'guest' && showPasswordChangeForm && (
  <div className="form-section active">
    <h2>Change Password</h2>

    {/* If password already changed, show confirmation screen */}
    {guestPasswordChanged ? (

```

```
<div className="guest-credentials">
```

```
<p className="guest-info">Your password has been updated:</p>
```

```
<div className="input-group readonly crt-box">
```

```
<User className="input-icon" size={20} />
```

```
<input type="text" value={guestCredentials.username} readOnly />
```

```
</div>
```

```
<div className="input-group readonly crt-box">
```

```
<Lock className="input-icon" size={20} />
```

```
<input
```

```
type={showGuestPassword ? 'text' : 'password'}
```

```
value={guestCredentials.password}
```

```
readOnly
```

```
/>
```

```
<button
```

```
type="button"
```

```
className="password-toggle"
```

```
onClick={() => setShowGuestPassword(!showGuestPassword)}
```

```
>
```

```
{showGuestPassword ? <EyeOff size={20} /> : <Eye size={20} />}
```

```
</button>
```

```
</div>
```

```
<button
```

```
className="primary-button"
```

```
onClick={handleContinueAfterGuestPasswordChange}
```

```
>
```

```
Continueea
```

```
</button>
```

```
</div>
```

```
): {
```

```
// Original password change form
```

```
<div className="guest-credentials">
```

```
  <p className="guest-info">Enter your new password for:  
{guestCredentials.username}</p>
```

```
<form onSubmit={handlePasswordChangeSubmit}>
```

```
  <div className="input-group">
```

```
    <Lock className="input-icon" size={20} />
```

```
    <input
```

```
      type={showNewPassword ? "text" : "password"}
```

```
      placeholder="New Password"
```

```
      value={passwordChangeData.newPassword}
```

```
      onChange={(e) =>
```

```
        setPasswordChangeData({ ...passwordChangeData, newPassword: e.target.value })
```

```
      }
```

```
      autoComplete="off"
```

```
      required
```

```
    />
```

```
  <button
```

```
    type="button"
```

```
    className="password-toggle"
```

```
    onClick={() => setShowNewPassword(!showNewPassword)}
```

```
  >
```

```
    {showNewPassword ? <EyeOff size={20} /> : <Eye size={20} />}
```

```
</button>
```

```
</div>
```



```

<div className="input-group">
  <Lock className="input-icon" size={20} />
  <input
    type={showConfirmNewPassword ? "text" : "password"}
    placeholder="Confirm New Password"
    value={passwordChangeData.confirmNewPassword}
    onChange={(e) =>
      setPasswordChangeData({ ...passwordChangeData, confirmNewPassword:
e.target.value })
    }
    autoComplete="off"
    required
  />
  <button
    type="button"
    className="password-toggle"
    onClick={() => setShowConfirmNewPassword(!showConfirmNewPassword)}
  >
    {showConfirmNewPassword ? <EyeOff size={20} /> : <Eye size={20} />}
  </button>
</div>

  <button type="submit" className="primary-button">Update Password</button>
</form>
</div>
)}

<div className="form-footer">
  {!guestPasswordChanged && (

```

```

    <button onClick={() => setShowPasswordChangeForm(false)} className="link-button">
      Cancel
    </button>
  )}
  <button onClick={() => switchView("login")} className="link-button">
    Back to Login
  </button>
</div>
</div>
)}

</div>
</div>
);
};

export default Login;

```

node/terminal.js

```

/* eslint-disable no-unused-vars */
import express from "express";
import mysql from "mysql2";
import cors from "cors";
import multer from "multer";
import path from "path";
import fs from "fs";

```

```
import { fileURLToPath } from "url";
import bcrypt from "bcrypt";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
app.use(cors());
app.use(express.json());

app.use("/uploads", express.static(path.join(__dirname, "uploads")));

const uploadDir = path.join(__dirname, "uploads", "user_picture");
if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true });
}

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, uploadDir);
  },
  filename: (req, file, cb) => {
    const uniqueName = Date.now() + "-" + Math.round(Math.random() * 1e9) +
path.extname(file.originalname);
    cb(null, uniqueName);
  }
});

const upload = multer({ storage });

const con = mysql.createConnection({
```

```

    host: "localhost",
    user: "root",
    password: "aanirudh_02",
    database: "CITDatabase"
  });

  con.connect(err => {
    if (err) {
      console.error("DB connection failed:", err);
      return;
    }
    console.log("✅ Connected to MySQL DB");
  });

  app.post("/api/register", upload.single("profilePhoto"), async (req, res) => {
    const { name, email, phone, dob, address, extra1, extra2, username, password } = req.body;

    if (!username || !password) {
      return res.status(400).send({ error: "Username and password required" });
    }

    try {
      const hashedPassword = await bcrypt.hash(password, 10);
      const profilePhoto = req.file ? path.join("user_picture", req.file.filename) : null;

      const sql = `INSERT INTO register
        (name, email, phone, dob, address, extra1, extra2, profile_photo, username, password)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`;

      con.query(sql,

```

```
[name, email, phone, dob, address, extra1, extra2, profilePhoto, username, hashedPassword],
```

```
    (err, result) => {  
      if (err) {  
        console.error("Register error:", err);  
        return res.status(500).send({ error: "Registration failed" });  
      }  
      res.send({ message: "Registration successful", profilePhoto });  
    }  
  );  
} catch (err) {  
  console.error("Hashing error:", err);  
  res.status(500).send({ error: "Failed to hash password" });  
}  
});
```

```
app.post("/api/anonymous", async (req, res) => {  
  const { username, password } = req.body;  
  
  if (!username || !password) {  
    return res.status(400).send({ error: "Username and password required" });  
  }  
  
  try {  
    const hashedPassword = await bcrypt.hash(password, 10);  
    const sql = `  
      INSERT INTO anonymous (username, password)  
      VALUES (?, ?)  
      ON DUPLICATE KEY UPDATE password = VALUES(password)  
    `;  
  }
```

```

con.query(sql, [username, hashedPassword], (err, result) => {
  if (err) {
    console.error("Anonymous insert error:", err);
    return res.status(500).send({ error: "Failed to save anonymous user" });
  }
  res.send({ message: "Anonymous user saved" });
});
} catch (err) {
  console.error("Hashing error:", err);
  res.status(500).send({ error: "Failed to hash password" });
}
});

app.post("/api/login", (req, res) => {
  const { username, password } = req.body;

  if (!username || !password) {
    return res.status(400).send({ error: "Username and password required" });
  }

  con.query("SELECT * FROM register WHERE username = ?", [username], async (err, results)
=> {
    if (err) return res.status(500).send({ error: "Database error" });

    if (results.length > 0) {
      const user = results[0];
      try {
        const match = await bcrypt.compare(password, user.password);
        if (match) {
          const safeUser = { ...user };

```

```

    delete safeUser.password;

    return res.send({ userType: "register", user: safeUser });
  } else {
    return res.status(401).send({ error: "Invalid credentials" });
  }
} catch (err) {
  console.error("Password comparison error:", err);
  return res.status(500).send({ error: "Password comparison error" });
}
}

```

```

con.query("SELECT * FROM anonymous WHERE username = ?", [username], async (err2,
results2) => {

```

```

  if (err2) return res.status(500).send({ error: "Database error" });

```

```

  if (results2.length > 0) {

```

```

    const user = results2[0];

```

```

    try {

```

```

      const match = await bcrypt.compare(password, user.password);

```

```

      if (match) {

```

```

        const safeUser = { ...user };

```

```

        delete safeUser.password;

```

```

        return res.send({ userType: "anonymous", user: safeUser });

```

```

      } else {

```

```

        return res.status(401).send({ error: "Invalid credentials" });

```

```

      }

```

```

    } catch (err) {

```

```

      console.error("Password comparison error:", err);

```

```

      return res.status(500).send({ error: "Password comparison error" });

```

```

    }

```

```

    } else {
        return res.status(401).send({ error: "Invalid credentials" });
    }
});

});

});

// replace your existing /api/anonymous-post route with this block
app.post("/api/anonymous-post", (req, res) => {
    const { anonUser, title, description, category } = req.body;

    if (!title || !description || !category) {
        return res.status(400).json({ error: "Title, description, and category are required." });
    }

    const user = typeof anonUser === "string" ? anonUser : (anonUser?.username || "Anonymous");

    const sql = `
        INSERT INTO anonymous_post (anonUser, title, description, category)
        VALUES (?, ?, ?, ?)
    `;

    con.query(sql, [user, title, description, category], (err, result) => {
        if (err) {
            console.error("Failed to insert anonymous_post:", err.code, err.sqlMessage);
            return res.status(500).json({ error: "Failed to save post" });
        }

        const insertedId = result.insertId;

```



```

// Return the inserted row so frontend has the exact data
con.query("SELECT * FROM anonymous_post WHERE id = ?", [insertedId], (selErr, rows) =>
{
  if (selErr) {
    console.error("Failed to fetch inserted post:", selErr);
    // still return insertId so client can proceed
    return res.status(201).json({ message: "Post saved", postId: insertedId });
  }

  const insertedRow = rows[0] || null;
  return res.status(201).json({
    message: "Post saved successfully",
    postId: insertedId,
    post: insertedRow
  });
});
});
});
});

app.get("/api/users", (req, res) => {
  const sql = `
    SELECT id, name, email, phone, dob, address, extra1, extra2, profile_photo, username
    FROM register
  `;

  con.query(sql, (err, results) => {
    if (err) {

```

```
    console.error("Error fetching users:", err);
    return res.status(500).send({ error: "Failed to fetch users" });
  }
}
```

```
const formattedResults = results.map(user => {
  const dobFormatted = user.dob
    ? new Date(user.dob).toISOString().split("T")[0]
    : "N/A";
```

```
//  Handle empty, null, or undefined address
```

```
const addressFormatted =
  user.address && user.address.trim() !== ""
    ? user.address
    : "Not mentioned";
```

```
return {
  ...user,
  dob: dobFormatted,
  address: addressFormatted,
};
});
```

```
res.send(formattedResults);
});
});
```

```
const PORT = 3001;
app.listen(PORT, () => {
```

```
console.log(`🚀 Server started on port ${PORT}`);  
});
```

## OUTPUT:-

  
**Welcome**

**Create Account - Step 1**







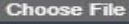
  



 ☐ I got to know through my friends

 ☐ I want to get clarity of mind

**Profile Photo (Optional):**



**Next Step**

[Back to Login](#)

✓ TERMINAL

JavaSE-

VITE v7.1.3 ready in 331 ms

→ Local: http://localhost:5173/

→ Network: use --host to expose

→ press h + enter to show help

Active code page: 65001

E:\React\_Projects\consultation>node server/index.js

🚀 Server started on port 3001

✅ Connected to MySQL DB

| Result Grid        |    |               |                         |            |            |                                     |
|--------------------|----|---------------|-------------------------|------------|------------|-------------------------------------|
| Filter Rows:       |    |               |                         |            |            |                                     |
| Edit:              |    |               |                         |            |            |                                     |
| Export/Import:     |    |               |                         |            |            |                                     |
| Wrap Cell Content: |    |               |                         |            |            |                                     |
|                    | id | name          | email                   | phone      | dob        | address                             |
| ▶                  | 1  | Karthik Raja  | karthikraja@gmail.com   | 9090123210 | 2003-01-22 |                                     |
|                    | 2  | Aanirudh      | aanirudhch@gmail.com    | 7010186524 | 2005-09-27 | 112 Gkr Nagar ,Chinniamapalayam,Coi |
|                    | 3  | Gajendran     | gajendran04@gmail.com   | 9010123031 | 2004-05-20 |                                     |
|                    | 4  | Guru Lakshman | gurulaksham03@gmail.com | 9090123456 | 2004-04-20 |                                     |

### **AIM:-**

Develop DJANGO web applications using URLs, Views, Templates and a MODEL to

retrieve data from the database created in MYSQL.

Design the page using bootstrap to display the content in the webpage.

### **CODE:-**

\Step 1: Setup Django Project

```
django-admin startproject student_portal
```

```
cd student_portal
```

```
python manage.py startapp students
```

Step 2: Configure MySQL in settings.py

# student\_portal/settings.py

```
DATABASES = {
```

```
    'default': {
```

```
        'ENGINE': 'django.db.backends.mysql',
```

```
        'NAME': 'student_db',      # Your MySQL database name
```

```
        'USER': 'root',           # MySQL username
```

```
        'PASSWORD': 'password_here', # MySQL password
```

```
'HOST': 'localhost',  
'PORT': '3306',  
}  
}  
  
INSTALLED_APPS = [  
    # ...  
    'students', # Add the app here  
]
```

### Step 3: Create a Model

```
# students/models.py  
from django.db import models  
  
class Student(models.Model):  
    first_name = models.CharField(max_length=50)  
    last_name = models.CharField(max_length=50)  
    email = models.EmailField()  
    course = models.CharField(max_length=100)  
  
    def __str__(self):  
        return f'{self.first_name} {self.last_name}'
```

### Step 4: Make Migrations

```
python manage.py makemigrations  
python manage.py migrate
```

### Step 5: Create a View

```
# students/views.py

from django.shortcuts import render

from .models import Student


def student_list(request):
    students = Student.objects.all()
    return render(request, 'students/student_list.html', {'students': students})
```

### Step 6: Setup URL

```
# student_portal/urls.py

from django.contrib import admin
from django.urls import path
from students import views


urlpatterns = [
    path('admin/', admin.site.urls),
    path('students/', views.student_list, name='student_list'),
]
```

### Step 7: Create Template

```
<!-- students/templates/students/student_list.html -->

<!DOCTYPE html>

<html lang="en">

<head>
```

```
<meta charset="UTF-8">

<title>Student Info Portal</title>

<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.cs
s" rel="stylesheet">

</head>

<body>

<div class="container mt-5">

  <h1 class="text-center mb-4">Student Information</h1>

  <table class="table table-bordered table-striped">

    <thead class="table-dark">

      <tr>

        <th>ID</th>

        <th>First Name</th>

        <th>Last Name</th>

        <th>Email</th>

        <th>Course</th>

      </tr>

    </thead>

    <tbody>

      {% for student in students %}

      <tr>

        <td>{{ student.id }}</td>

        <td>{{ student.first_name }}</td>

        <td>{{ student.last_name }}</td>

        <td>{{ student.email }}</td>

        <td>{{ student.course }}</td>

      </tr>

    </tbody>

  </table>

</div>

</body>

</html>
```



```
</tr>
{% empty %}
<tr>
    <td colspan="5" class="text-center">No students found</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
</body>
</html>
```

#### Step 8: Run the Server

python manage.py runserver

Visit: <http://127.0.0.1:8000/students/>

You will see a Bootstrap-styled table displaying all students from your MySQL database.

**OUTPUT:-**

**Student Information**

| ID | First Name | Last Name | Email                     | Course      |
|----|------------|-----------|---------------------------|-------------|
| 1  | JOHN       | DOE       | john.doe@example.com      | Mathematics |
| 2  | JANE       | SMITH     | jane.smith@example.com    | Physics     |
| 3  | ALICE      | JOHNSON   | alice.johnson@example.com | Chemistry   |
| 4  | BOB        | WILLIAMS  | bob.williams@example.com  | Biology     |

---

### **AIM:-**

Create a Django web app to manage student data (view, add, update, delete) using Python lists and dictionaries instead of a database, with navigation across multiple HTML pages.

### **CODE:-**

Create a Django web app to manage student data (view, add, update, delete) using Python lists and dictionaries instead of a database, with navigation across multiple HTML pages.

---

#### Step-by-Step Implementation

##### **1** Project Setup

```
django-admin startproject student_project
cd student_project
python manage.py startapp students
```

---

##### **2** students/views.py

```
from django.shortcuts import render, redirect
```

```
# Simulated student data
```

```
students = [
    {"id": 1, "name": "Alice", "roll": "101", "course": "Math"},

```

```
    {"id": 2, "name": "Bob", "roll": "102", "course": "Physics"},  
]
```

```
def home(request):  
    return render(request, 'students/home.html')
```

```
def student_list(request):  
    return render(request, 'students/student_list.html', {"students": students})
```

```
def add_student(request):  
    if request.method == "POST":  
        name = request.POST.get("name")  
        roll = request.POST.get("roll")  
        course = request.POST.get("course")  
        if name and roll and course:  
            new_id = max([s["id"] for s in students] or [0]) + 1  
            students.append({"id": new_id, "name": name, "roll": roll, "course":  
course})  
            return redirect('student_list')  
        return render(request, 'students/add_student.html')
```

```
def update_student(request, student_id):  
    student = next((s for s in students if s["id"] == student_id), None)  
    if not student:  
        return redirect('student_list')  
  
    if request.method == "POST":
```

```
student["name"] = request.POST.get("name")
student["roll"] = request.POST.get("roll")
student["course"] = request.POST.get("course")
return redirect('student_list')
```

```
return render(request, 'students/update_student.html', {"student": student})
```

```
def delete_student(request, student_id):
    global students
    students = [s for s in students if s["id"] != student_id]
    return redirect('student_list')
```

---

### 3 students/urls.py

```
from django.urls import path
from . import views
```

```
urlpatterns = [
    path('', views.home, name='home'),
    path('students/', views.student_list, name='student_list'),
    path('students/add/', views.add_student, name='add_student'),
    path('students/update/<int:student_id>/', views.update_student,
name='update_student'),
    path('students/delete/<int:student_id>/', views.delete_student,
name='delete_student'),
]
```

---

### 4 student\_project/urls.py

```
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('students.urls')),
]
```

---

## 5 Templates (students/templates/students/)

home.html

```
<!DOCTYPE html>
<html>
<head><title>Home</title></head>
<body>
<h1>Student Management System</h1>
<a href="{% url 'student_list' %}">View Students</a> |
<a href="{% url 'add_student' %}">Add Student</a>
</body>
</html>
```

student\_list.html

```
<!DOCTYPE html>
<html>
<head><title>Student List</title></head>
<body>
<h1>Students</h1>
<a href="{% url 'home' %}">Home</a> |
```

```

<a href="{% url 'add_student' %}">Add Student</a>
<table border="1">
<tr><th>ID</th><th>Name</th><th>Roll</th><th>Course</th><th>Actions</t
h></tr>
{% for student in students %}
<tr>
<td>{{ student.id }}</td>
<td>{{ student.name }}</td>
<td>{{ student.roll }}</td>
<td>{{ student.course }}</td>
<td>
<a href="{% url 'update_student' student.id %}">Edit</a> |
<a href="{% url 'delete_student' student.id %}">Delete</a>
</td>
</tr>
{% endfor %}
</table>
</body>
</html>
add_student.html
<!DOCTYPE html>
<html>
<head><title>Add Student</title></head>
<body>
<h1>Add Student</h1>
<a href="{% url 'student_list' %}">Back to List</a>
<form method="POST">{% csrf_token %}

```

```

<label>Name:</label><input type="text" name="name"><br>
<label>Roll:</label><input type="text" name="roll"><br>
<label>Course:</label><input type="text" name="course"><br>
<input type="submit" value="Add Student">
</form>
</body>
</html>
update_student.html
<!DOCTYPE html>
<html>
<head><title>Update Student</title></head>
<body>
<h1>Update Student</h1>
<a href="{% url 'student_list' %}">Back to List</a>
<form method="POST">{% csrf_token %}
<label>Name:</label><input type="text" name="name" value="{{
student.name }}"><br>
<label>Roll:</label><input type="text" name="roll" value="{{ student.roll
 }}"><br>
<label>Course:</label><input type="text" name="course" value="{{
student.course }}"><br>
<input type="submit" value="Update Student">
</form>
</body>
</html>

```



- View, add, update, delete students.
- Uses Python list & dictionaries (no DB).
- Navigation via links/menu.
- Form validation using Django CSRF and simple checks.

## OUTPUT:-

[Student List](#) [Add Student](#)

### Add Student

Name:

Andrea

Roll:

02

Course:

Software Systems

[Add Student](#)

[Cancel](#)

---

## Student Management System

[Student List](#) [Add Student](#)

### Students

| ID | Name   | Roll | Course           | Actions              |                        |
|----|--------|------|------------------|----------------------|------------------------|
| 1  | Alice  | 101  | Math             | <a href="#">Edit</a> | <a href="#">Delete</a> |
| 2  | Bob    | 102  | Physics          | <a href="#">Edit</a> | <a href="#">Delete</a> |
| 3  | Andrea | 02   | Software Systems | <a href="#">Edit</a> | <a href="#">Delete</a> |

---